

PART 3

APPLICATION DESIGN AND DEVELOPMENT

One of the key requirements of the relational model is that data values be atomic: multivalued, composite, and other complex data types are disallowed by the core relational model. However, there are many applications where the constraints on data types imposed by the relational model cause more problems than they solve. In Chapter 8, we discuss several *complex data types*, including semistructured data types that are widely used in building applications, object-based data, textual data, and spatial data.

Practically all use of databases occurs from within application programs. Correspondingly, almost all user interaction with databases is indirect, via application programs. Database-backed applications are ubiquitous on the web as well as on mobile platforms. In Chapter 9, we study tools and technologies that are used to build applications, focusing on interactive applications that use databases to store and retrieve data.

CHAPTER 8



Complex Data Types

The relational model is very widely used for data representation for a large number of application domains. One of the key requirements of the relational model is that data values be atomic: multivalued, composite, and other complex data types are disallowed by the core relational model. However, there are many applications where the constraints on data types imposed by the relational model cause more problems than they solve. In this chapter, we discuss several *non-atomic data types* that are widely used, including semi-structured data, object-based data, textual data, and spatial data.

8.1 Semi-structured Data

Relational database designs have tables with a fixed number of attributes, each of which contains an atomic value. Changes to the schema, such as adding an extra attribute, are rare events, and may require changing of application code. Such a design is well suited to many organizational applications.

However, there are many application domains that need to store more complex data, whose schema changes often. Fast evolving web applications are an example of such a domain. As an example of the data management needs of such applications, consider the profile of a user which needs to be accessible to a number of different applications. The profile contains a variety of attributes, and there are frequent additions to the attributes stored in the profile. Some attributes may contain complex data; for example, an attribute may store a set of interests that can be used to show the user articles related to the set of interests. While such a set can be stored in a normalized fashion in a separate relation, a set data type allows significantly more efficient access than does a normalized representation. We study a number of data models that support representation of semi-structured data in this section.

Data exchange is another very important motivation for semi-structured data representations; it is perhaps even more important than storage for many applications. A popular architecture for building information systems today is to create a web service that allows retrieval of data and to build application code that displays the data and allows user interaction. Such application code may be developed as mobile applications,

or it may be written in JavaScript and run on the browser. In either case, the ability to run on the client's machine allows developers to create very responsive user interfaces, unlike the early generation of web interfaces where backend servers send HTML marked-up text to browsers, which display the HTML. A key to building such applications is the ability to efficiently exchange and process complex data between backend servers and clients. We study the JSON and XML data models that have been widely adopted for this task.

8.1.1 Overview of Semi-structured Data Models

The relational data model has been extended in several ways to support the storage and data exchange needs of modern applications.

8.1.1.1 Flexible Schema

Some database systems allow each tuple to potentially have a different set of attributes; such a representation is referred to as a **wide column** data representation. The set of attributes is not fixed in such a representation; each tuple may have a different set of attributes, and new attributes may be added as needed.

A more restricted form of this representation is to have a fixed but very large number of attributes, with each tuple using only those attributes that it needs, leaving the rest with null values; such a representation is called a **sparse column** representation.

8.1.1.2 Multivalued Data Types

Many data representations allow attributes to contain non-atomic values. Many databases allow the storage of **sets**, **multisets**, or **arrays** as attribute values. For example, an application that stores topics of interest to a user, and uses the topics to target articles or advertisements to the user, may store the topics as a set. An example of such a set may be:

{ basketball, La Liga, cooking, anime, Jazz }

Although a set-valued attribute can be stored in a normalized form as we saw earlier in Section 6.7.2, doing so provides no benefits in this case, since lookups are always based on the user, and normalization would significantly increase the storage and querying overhead.

Some representations allow attributes to store *key-value maps*, which store key-value pairs. A **key-value map**, often just called a **map**, is a set of (*key, value*) pairs, such that each key occurs in at most one element. For example, e-commerce sites often list specifications or details for each product that they sell, such as brand, model, size, color, and numerous other product-specific details. The set of specifications may be different for each product. Such specifications can be represented as a map, where

the specifications form the key, and the associated value is stored with the key. The following example illustrates such a map:

{ (brand, Apple), (ID, MacBook Air), (size, 13), (color, silver) }

The `put(key, value)` method can be used to add a key-value pair, while the `get(key)` method can be used to retrieve the value associated with a key. The `delete(key)` method can be used to delete a key-value pair from the map.

Arrays are very important for scientific and monitoring applications. For example, scientific applications may need to store images, which are basically two-dimensional arrays of pixel values. Scientific experiments as well as industrial monitoring applications often use multiple sensors that provide readings at regular intervals. Such readings can be viewed as an array. In fact, treating a stream of readings as an array requires far less space than storing each reading as a separate tuple, with attributes such as (*time*, *reading*). Not only do we avoid storing the *time* attribute explicitly (it can be inferred from the offset), but we can also reduce per-tuple overhead in the database, and most importantly we can use compression techniques to reduce the space needed to store an array of readings.

Support for multivalued attribute types was proposed early in the history of databases, and the associated data model was called the *non first-normal-form*, or *NFNF*, data model. Several relational databases such as Oracle and PostgreSQL support set and array types.

An **array database** is a database that provides specialized support for arrays, including efficient compressed storage, and query language extensions to support operations on arrays. Examples include the Oracle GeoRaster, the PostGIS extension to PostgreSQL, the SciQL extension of MonetDB, and SciDB, a database tailored for scientific applications, with a number of features tailored for array data types.

8.1.1.3 Nested Data Types

Many data representations allow attributes to be structured, directly modeling composite attributes in the E-R model. For example, an attribute *name* may have component attributes *firstname*, and *lastname*. These representations also support multivalued data types such as sets, arrays, and maps. All of these data types represent a hierarchy of data types, and that structure leads to the use of the term **nested data types**. Many databases support such types as part of their support for object-oriented data, which we describe in Section 8.2.

In this section, we outline two widely used data representations that allow values to have complex internal structures and that are flexible in that values are not forced to adhere to a fixed schema. These are the **JavaScript Object Notation** (**JSON**), which we describe in Section 8.1.2, and the **Extensible Markup Language** (**XML**), which we describe in Section 8.1.3.

Like the wide-table approach, the JSON and XML representations provide flexibility in the set of attributes that a record contains, as well as the types of these attributes.

However, the JSON and XML representations permit a more flexible structuring of data, where objects could have sub-objects; each object thus corresponds to a tree structure.

Since they allow multiple pieces of information about a business object to be packaged into a single structure, the JSON and XML representations have both found significant acceptance in the context of data exchange between applications.

Today, JSON is widely used today for exchanging data between the backends and the user-facing sides of applications, such as mobile apps, and Web apps. JSON has also found favor for storing complex objects in storage systems that collect different data related to a particular user into one large object (sometimes referred to as a document), allowing data to be retrieved without the need for joins. XML is an older representation and is used by many systems for storing configuration and other information, and for data exchange.

8.1.1.4 Knowledge Representation

Representation of human knowledge has long been a goal of the artificial intelligence community. A variety of models were proposed for this task, with varying degrees of complexity; these could represent facts as well as rules about facts. With the growth of the web, a need arose to represent extremely large knowledge bases, with potentially billions of facts. The *Resource Description Format* (*RDF*) data representation is one such representation that has found very wide acceptance. The representation actually has far fewer features than earlier representations, but it was better suited to handle very large data volumes than the earlier knowledge representations.

Like the E-R model which we studied earlier, RDF models data as objects that have attributes and have relationships with other objects. RDF data can be viewed as a set of triples (3-tuples), or as a graph, with objects and attribute values modeled as nodes and relationships and attribute names as edges. We study RDF in more detail in Section 8.1.4.

8.1.2 JSON

The **JavaScript Object Notation** (**JSON**), is a textual representation of complex data types that is widely used to transmit data between applications and to store complex data. JSON supports the primitive data types integer, real and string, as well as arrays, and "objects," which are a collection of (attribute name, value) pairs.

Figure 8.1 shows an example of data represented using JSON. Since objects do not have to adhere to any fixed schema, they are basically the same as key-value maps, with the attribute names as keys and the attribute values as the associated values.

The example also illustrates arrays, shown in square brackets. In JSON, an array can be thought of as a map from integer offsets to values, with the square-bracket syntax viewed as just a convenient way of creating such maps.

JSON is today *the* primary data representation used for communication between applications and web services. Many modern applications use web services to store

```
{
    "ID": "22222",
    "name": {
        "firstname: "Albert",
        "lastname: "Einstein"
    },
    "deptname": "Physics",
    "children": [
        {"firstname": "Hans", "lastname": "Einstein" },
        {"firstname": "Eduard", "lastname": "Einstein" }
    ]
}
```

Figure 8.1 Example of JSON data.

and retrieve data and to perform computations at a backend server; web services are described in more detail in Section 9.5.2. Applications invoke web services by sending parameters either as simple values such as strings or numbers, or by using JSON for more complex parameters. The web service then returns results using JSON. For example, an email user interface may invoke web services for each of these tasks: authenticating the user, fetching email header information to show a list of emails, fetching an email body, sending email, and so on.

The data exchanged in each of these steps are complex and have an internal structure. The ability of JSON to represent complex structures, and its ability to allow flexible structuring, make it a good fit for such applications.

A number of libraries are available that make it easy to transform data between the JSON representation and the object representation used in languages such as JavaScript, Java, Python, PHP, and other languages. The ease of interfacing between JSON and programming language data structures has played a significant role in the widespread use of JSON.

Unlike a relational representation, JSON is verbose and takes up more storage space for the same data. Further, parsing the text to retrieve required fields can be very CPU intensive. Compressed representations that also make it easier to retrieve values without parsing are therefore popular for storage of data. For example, a compressed binary format called BSON (short for Binary JSON) is used in many systems for storing JSON data.

The SQL language itself has been extended to support the JSON representation in several ways:

- JSON data can be stored as a JSON data type.
- SQL queries can generate JSON data from relational data:

- There are SQL extensions that allow construction of JSON objects in each row of a query result. For example, PostgreSQL supports a json_build_object() function. As an example of its use, json_build_object('ID', 12345, 'name' 'Einstein') returns a JSON object {"ID": 12345, "name", "Einstein"}.
- There are also SQL extensions that allow creation of a JSON object from a collection of rows by using an aggregate function. For example, the json_agg aggregate function in PostgreSQL allows creation of a single JSON object from a collection of JSON objects. Oracle supports a similar aggregate function json_objectagg, as well as an aggregate json_arraytagg, which creates a JSON array with objects in a specified order. SQL Server supports a FOR JSON AUTO clause that formats the result of an SQL query as a JSON array, with one element per row in the SQL query.

- SQL queries can extract data from a JSON object using some form of path constructs. For example, in PostgreSQL, if a value *v* is of type JSON and has an attribute "ID", *v*–>'ID' would return the value of the "ID" attribute of *v*. Oracle supports a similar feature, using a "." instead of "–>", while SQL Server uses a function JSON_VALUE(value, path) to extract values from JSON objects using a specified path.

The exact syntax and semantics of these extensions, unfortunately, depend entirely on the specific database system. You can find references to more details on these extensions in the bibliographic notes for this chapter, available online.

8.1.3 XML

The XML data representation adds **tags** enclosed in angle brackets, <>, to mark up information in a textual representation. Tags are used in pairs, with <tag> and </tag> delimiting the beginning and the end of the portion of the text to which the tag refers. For example, the title of a document might be marked up as follows:

```
<title>Database System Concepts</title>
```

Such tags can be used to represent relational data specifying relation names and attribute names as tags, as shown below:

```
<course>
    <course_id> CS-101 </course_id>
    <title> Intro. to Computer Science </title>
    <dept_name> Comp. Sci. </dept_name>
    <credits> 4 </credits>
</course>
```

```
<purchase_order>
    <identifier> P-101 </identifier>
    <purchaser>
        <name> Cray Z. Coyote </name>
        <address> Route 66, Mesa Flats, Arizona 86047, USA  </address>
    </purchaser>
    <supplier>
        <name> Acme Supplies  </name>
        <address> 1 Broadway, New York, NY, USA  </address>
    </supplier>
    <itemlist>
        <item>
            <identifier>  RS1  </identifier>
            <description> Atom powered rocket sled  </description>
            <quantity> 2 </quantity>
            <price> 199.95 </price>
        </item>
        <item>
            <identifier>  SG2  </identifier>
            <description> Superb glue  </description>
            <quantity> 1 </quantity>
            <unit-of-measure> liter </unit-of-measure>
            <price> 29.95 </price>
        </item>
    </itemlist>
    <total_cost> 429.85 </total_cost>
    <payment_terms> Cash-on-delivery </payment_terms>
    <shipping_mode> 1-second-delivery </shipping_mode>
</purchase_order>
```

Figure 8.2 XML representation of a purchase order.

Unlike with a relational schema, new tags can be introduced easily, and with suitable names the data are "self-documenting" in that a human can understand or guess what a particular piece of data means based on the name.

Furthermore, tags can be used to create hierarchical structures, which is not possible with the relational model. Hierarchical structures are particularly important for representing business objects that must be exchanged between organizations; examples include bills, purchase orders, and so forth.

Figure 8.2, which shows how information about a purchase order can be represented in XML, illustrates a more realistic use of XML. Purchase orders are typically generated by one organization and sent to another. A purchase order contains a variety of information; the nested representation allows all information in a purchase order to

be represented naturally in a single document. (Real purchase orders have considerably more information than that depicted in this simplified example.) XML provides a standard way of tagging the data; the two organizations must of course agree on what tags appear in the purchase order and what they mean.

The XQuery language was developed to support querying of XML data. Further details of XML and XQuery may be found in Chapter 30. Although XQuery implementations are available from several vendors, unlike SQL, adoption of XQuery has been relatively limited.

However, the SQL language itself has been extended to support XML in several ways:

- XML data can be stored as an XML data type.
- SQL queries can generate XML data from relational data. Such extensions are very useful for packaging related pieces of data into one XML document, which can then be sent to another application.
  The extensions allow the construction of XML representations from individual rows, as well as the creation of an XML document from a collection of rows by using an XMLAGG aggregate function.
- SQL queries can extract data from an XML data type value. For example, the XPath language supports "path expressions" that allow the extraction of desired parts of data from an XML document.

You can find more details on these extensions in Chapter 30.

8.1.4 RDF and Knowledge Graphs

The **Resource Description Framework** (**RDF**) is a data representation standard based on the entity-relationship model. We provide an overview of RDF in this section.

8.1.4.1 Triple Representation

The RDF model represents data by a set of **triples** that are in one of these two forms:

1. (*ID, attribute-name, value*)
2. (*ID1, relationship-name, ID2*)

where *ID, ID1* and *ID2* are identifiers of entities; entities are also referred to as **resources** in RDF. Note that unlike the E-R model, the RDF model only supports binary relationships, and it does not support more general n-ary relationships; we return to this issue later.

The first attribute of a triple is called its **subject**, the second attribute is called its **predicate**, and the last attribute is called its **object**. Thus, a triple has the structure (subject, predicate, object).

```
10101       instance-of    instructor .
10101       name           "Srinivasan" .
10101       salary         "6500" .
00128       instance-of    student .
00128       name           "Zhang" .
00128       tot_cred       "102" .
comp_sci    instance-of    department .
comp_sci    dept_name      "Comp. Sci." .
biology     instance-of    department .
CS-101      instance-of    course .
CS-101      title          "Intro. to Computer Science" .
CS-101      course_dept    comp_sci .
sec1        instance-of    section .
sec1        sec_course     CS-101 .
sec1        sec_id         "1" .
sec1        semester       "Fall" .
sec1        year           "2017" .
sec1        classroom      packard-101 .
sec1        time_slot_id   "H" .
10101       inst_dept      comp_sci .
00128       stud_dept      comp_sci .
00128       takes          sec1 .
10101       teaches        sec1 .
```

Figure 8.3 RDF representation of part of the University database.

Figure 8.3 shows a triple representation of a small part of the University database. All attribute values are shown in quotes, while identifiers are shown without quotes. Attribute and relationship names (which form the predicate part of each triple) are also shown without quotes.

In our example, we use the ID values to identify instructors and students and *course_id* to identify courses. Each of their attributes is represented as a separate triple. The type information of objects is provided by the *instance-of* relationship; for example, 10101 is identified as an instance of instructor, while 00128 is an instance of student. To follow RDF syntax, the identifier of the Comp. Sci. department is denoted as comp_sci. Only one attribute of the department, *dept_name*, is shown. Since the primary key of *section* is composite, we have created new identifiers to identify sections; "sec1" identifies one such section, shown with its *semester*, *year* and *sec_id* attributes, and with a relationship *course* to CS-101.

Relationships shown in the figure include the *takes* and *teaches* relationships, which appear in the university schema. The departments of instructors, students and courses are shown as relationships *inst_dept*, *stud_dept* and *course_dept* respectively, following the E-R model; similarly, the classroom associated with a section is also shown as a *classroom* relationship with a classroom object (packard-101, in our example), and the

course associated with a section is shown as a relationship *sec_course* between the section and the course.

As we saw, entity type information is represented using *instance-of* relationships between entities and objects representing types; type-subtype relationships can also be represented as *subtype* edges between type objects.

In contrast to the E-R model and relational schemas, RDF allows new attributes to be easily added to an object and also to create new types of relationships.

8.1.4.2 Graph Representation of RDF

The RDF representation has a very natural graph interpretation. Entities and attribute values can be considered as nodes, and attribute names and relationships can be considered as edges between the nodes. The attribute/relationship name can be viewed as the label of the corresponding edge. Figure 8.4 shows a graph representation of the data from Figure 8.3. Objects are shown as ovals, attribute values in rectangles, and relationships as edges with associated labels identifying the relationship. We have omitted the *instance-of* relationships for brevity.

A representation of information using the RDF graph model (or its variants and extensions) is referred to as a **knowledge graph**. Knowledge graphs are used for a variety of purposes. One such application is to store facts that are harvested from a variety of data sources, such as Wikipedia, Wikidata, and other sources on the web. An example of a fact is "Washington, D.C. is the capital of U.S.A." Such a fact can be represented as an edge labeled *capital-of* connecting two nodes, one representing the entity Washington, D.C., and the other representing the entity U.S.A.

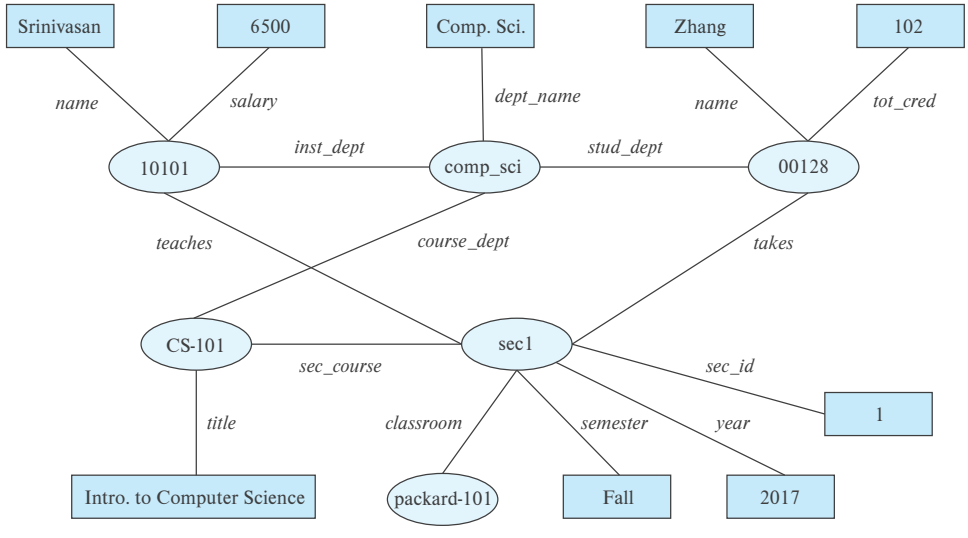


Figure 8.4 Graph representation of RDF data.

Questions about entities can be answered using a knowledge graph that contains relevant information. For example, the question "Which city is the capital of the U.S.A.?" can be answered by looking for an edge labeled *capital-of*, linking an entity to the country U.S.A. (If type information is available, the query may also verify that there is an *instance-of* edge connecting Washington, D.C., to a node representing the entity type *City*).

8.1.4.3 SPARQL

SPARQL is a query language designed to query RDF data. The language is based on triple patterns, which look like RDF triples but may contain variables. For example, the triple pattern:

?cid *title* "Intro. to Computer Science"

would match all triples whose predicate is "title" and object is "Intro. to Computer Science". Here, ?cid is a variable that can match any value.

Queries can have multiple triple patterns, with variables shared across triples. Consider the following pair of triples:

?cid *title* "Intro. to Computer Science"
?sid *course* ?cid

On the university-triple dataset shown in Figure 8.3, the first triple pattern matches the triple (CS-101, title, "Intro. to Computer Science"), while the second triple pattern matches (sec1, course, CS-101). The shared variable ?cid enforces a join condition between the two triple patterns.

We can now show a complete SPARQL query. The following query retrieves names of all students who have taken a section whose course is titled "Intro. to Computer Science".

```
select ?name
where {
        ?cid title "Intro. to Computer Science" .
        ?sid course ?cid .
        ?id takes ?sid .
        ?id name ?name .
}
```

The shared variables between these triples enforce a join condition between the tuples matching each of these triples.

Note that unlike in SQL, the predicate in a triple pattern can be a variable, which can match any relationship or attribute name. SPARQL has many more features, such

as aggregation, optional joins (similar to outerjoins), and subqueries. For more information in SPARQL, see the references in Further Reading.

8.1.4.4 Representing N-ary Relationships

Relationships represented as edges can model only binary relationships. Knowledge graphs have been extended to store more complex relationships. For example, knowledge graphs have been extended with temporal information to record the time period during which a fact is true; if the capital of the U.S.A. changed from Washington, DC., to say, New York, in 2050, this would be represented by two facts, one for the period ending in 2050 when Washington was the capital, and one for the period after 2050.

As we saw in Section 6.9.4, an *n*-ary relationship can be represented using binary relationships by creating an artificial entity corresponding to a tuple in an *n*-ary relationship and linking that artificial entity to each of the entities participating in the relationship. In the preceding example, we can create an artificial entity e_1 to represent the fact that Barack Obama was president of the U.S.A. from 2008 to 2016. We link e_1 to the entities representing Obama and U.S.A. by *person* and *country* relationship edges respectively, and to the values 2008 and 2016 by attribute edges *president-from* and *president-till* respectively. If we chose to represent years as entities, the edges created to the two years above would represent relationships instead of attributes.

The above idea is similar to the E-R model notion of aggregation which, as we saw in Section 6.8.5, can treat a relationship as an entity; this idea is called **reification** in RDF. Reification is used in many knowledge-graph representations, where the extra information such as time period of validity are treated as *qualifiers* of the underlying edge.

Other models add a fourth attribute, called the context, to triples; thus, instead of storing triples, they store **quads**. The basic relationship is still binary, but the fourth attribute allows a context entity to be associated with a relationship. Information such as valid time period can be treated as attributes of the context entity.

There are several knowledge bases, such as Wikidata, DBPedia, Freebase, and Yago, that provide an RDF/knowledge graph representation of a wide variety of knowledge. In addition, there are a very large number of domain-specific knowledge graphs. The **linked open data** project is aimed at making a variety of such knowledge graphs open source and further creating links between these independently created knowledge graphs. Such links allow queries to make inferences using information from multiple knowledge graphs along with links to the knowledge graphs. References to more information on this topic may be found in the bibliographic notes for this chapter, available online.

8.2 Object Orientation

The **object-relational data model** extends the relational data model by providing a richer type system, including complex data types and object orientation. Relational query

languages, in particular SQL, have been extended correspondingly to deal with the richer type system. Such extensions attempt to preserve the relational foundations—in particular, the declarative access to data—while extending the modeling power.

Many database applications are written using an object-oriented programming language, such as Java, Python, or C++, but they need to store and fetch data from databases. Due to the type difference between the native type system of the object-oriented programming language and the relational model supported by databases, data need to be translated between the two models whenever they are fetched or stored. Merely extending the type system supported by the database was not enough to solve this problem completely. Having to express database access using a language (SQL) that is different from the programming language again makes the job of the programmer harder. It is desirable, for many applications, to have programming language constructs or extensions that permit direct access to data in the database, without having to go through an intermediate language such as SQL.

Three approaches are used in practice for integrating object orientation with database systems:

1. Build an **object-relational database system**, which adds object-oriented features to a relational database system.
2. Automatically convert data from the native object-oriented type system of the programming language to a relational representation for storage, and vice versa for retrieval. Data conversion is specified using an **object-relational mapping**.
3. Build an **object-oriented database system**, that is, a database system that natively supports an object-oriented type system and allows direct access to data from an object-oriented programming language using the native type system of the language.

We provide a brief introduction to the first two approaches in this section. While the third approach, the object-oriented database approach, has some benefits over the first two approaches in terms of language integration, it has not seen much success for two reasons. First, declarative querying is very important for efficiently accessing data, and such querying is not supported by imperative programming languages. Second, direct access to objects via pointers was found to result in increased risk of database corruption due to pointer errors. We do not describe the object-oriented approach any further.

8.2.1 Object-Relational Database Systems

In this section, we outline how object-oriented features can be added to relational database systems.

8.2.1.1 User-Defined Types

Object extensions to SQL allow creation of structured user-defined types, references to such types, and tables containing tuples of such types.[1]

```
create type Person
        (ID varchar(20) primary key,
        name varchar(20),
         address varchar(20))
        ref from(ID);
create table people of Person;
```

We can create a new person as follows:

```
insert into people (ID, name, address) values
        ('12345', 'Srinivasan', '23 Coyote Run');
```

Many database systems support array and table types; attributes of relations and of user-defined types can be declared to be of such array or table types. The support for such features as well as the syntax varies widely by database system. In PostgreSQL, for example, *integer*[] denotes an array of integers whose size is not prespecified, while Oracle supports the syntax **varray**(10) **of integer** to specify an array of 10 integers. SQL Server allows table-valued types to be declared as shown in the following example:

```
create type interest as table (
        topic varchar(20),
        degree_of_interest int
);
create table users (
        ID varchar(20),
        name varchar(20),
        interests interest
);
```

User-defined types can also have methods associated with them. Only a few database systems, such as Oracle, support this feature; we omit details.

8.2.1.2 Type Inheritance

Consider the earlier definition of the type *Person* and the table *people*. We may want to store extra information in the database about people who are students and about people

[1] Structured types are different from the simpler "distinct" data types that we covered in Section 4.5.5.

who are teachers. Since students and teachers are also people, we can use inheritance to define the student and teacher types in SQL:

```
create type Student under Person
    (degree varchar(20)) ;
create type Teacher under Person
    (salary integer);
```

Both *Student* and *Teacher* inherit the attributes of *Person*—namely, *ID*, *name*, and *address*. *Student* and *Teacher* are said to be subtypes of *Person*, and *Person* is a supertype of *Student*, as well as of *Teacher*.

Methods of a structured type are inherited by its subtypes, just as attributes are. However, a subtype can redefine the effect of a method. We omit details.

8.2.1.3 Table Inheritance

Table inheritance allows a table to be declared as a subtable of another table and corresponds to the E-R notion of specialization/generalization. Several database systems support table inheritance, but in different ways.

In PostgreSQL, we could create a table *people* and then create tables *students* and *teachers* as subtables of *people* as follows:

```
create table students
    (degree varchar(20))
    inherits people;
create table teachers
    (salary integer)
    inherits people;
```

As a result, every attribute present in the table *people* is also present in the subtables *students* and *teachers*.

SQL:1999 supports table inheritance but requires table types to be specified first. Thus, in Oracle, which supports SQL:1999, we could use:

```
create table people of Person;
create table students of Student
    under people;
create table teachers of Teacher
    under people;
```

where the types *Student* and *Teacher* have been declared to be subtypes of *Person* as described earlier.

In either case, we can insert a tuple into the *student* table as follows:

```
insert into student values ('00128', 'Zhang', '235 Coyote Run', 'Ph.D.');
```

where we provide values for the attributes inherited from *people* as well as the local attributes of *student*.

When we declare *students* and *teachers* as subtables of *people*, every tuple present in *students* or *teachers* becomes implicitly present in *people*. Thus, if a query uses the table *people*, it will find not only tuples directly inserted into that table but also tuples inserted into its subtables, namely, *students* and *teachers*. However, only those attributes that are present in *people* can be accessed by that query. SQL permits us to find tuples that are in *people* but not in its subtables by using "**only** *people*" in place of *people* in a query.

8.2.1.4 Reference Types in SQL

Some SQL implementations such as Oracle support reference types. For example, we could define the *Person* type as follows, with a reference-type declaration:

```
create type Person
        (ID varchar(20) primary key,
        name varchar(20),
         address varchar(20))
        ref from(ID);
create table people of Person;
```

By default, SQL assigns system-defined identifiers for tuples, but an existing primary-key value can be used to reference a tuple by including the **ref from** clause in the type definition as shown above.

We can define a type *Department* with a field *name* and a field *head* that is a reference to the type *Person*. We can then create a table *departments* of type *Department*, as follows:

```
create type Department (
        dept_name varchar(20),
        head ref(Person) scope people);
create table departments of Department;
```

Note that the **scope** clause above completes the definition of the foreign key from *departments.head* to the *people* relation.

When inserting a tuple for *departments*, we can then use:

```
insert into departments
        values ('CS', '12345');
```

since the ID attribute is used as a reference to *Person*. Alternatively, the definition of *Person* can specify that the reference must be generated automatically by the system when a *Person* object is created. System-generated identifiers can be retrieved using **ref**(*r*) where *r* is a table name of table alias used in a query. Thus, we could create a *Person* tuple, and, using the ID or name of the person, we could retrieve the reference to the tuple in a subquery, which is used to create the value for the *head* attribute when inserting a tuple into the *departments* table. Since most database systems do not allow subqueries in an **insert into** *departments* **values** statement, the following two queries can be used to carry out the task:

```
insert into departments
    values ('CS', null);
update departments
    set head = (select ref(p)
        from people as p
        where ID = '12345')
    where dept_name = 'CS';
```

References are dereferenced in SQL:1999 by the −> symbol. Consider the *departments* table defined earlier. We can use this query to find the names and addresses of the heads of all departments:

```
select head−>name, head−>address
from departments;
```

An expression such as "*head*−>*name*" is called a **path expression**.

Since *head* is a reference to a tuple in the *people* table, the attribute *name* in the preceding query is the *name* attribute of the tuple from the *people* table. References can be used to hide join operations; in the preceding example, without the references, the *head* field of *department* would be declared a foreign key of the table *people*. To find the name and address of the head of a department, we would require an explicit join of the relations *departments* and *people*. The use of references simplifies the query considerably.

We can use the operation **deref** to return the tuple pointed to by a reference and then access its attributes, as shown below:

```
select deref(head).name
from departments;
```

8.2.2 Object-Relational Mapping

Object-relational mapping (**ORM**) systems allow a programmer to define a mapping between tuples in database relations and objects in the programming language.

An object, or a set of objects, can be retrieved based on a selection condition on its attributes; relevant data are retrieved from the underlying database based on the selection conditions, and one or more objects are created from the retrieved data, based on the prespecified mapping between objects and relations.

The program can update retrieved objects, create new objects, or specify that an object is to be deleted, and then issue a save command; the mapping from objects to relations is then used to correspondingly update, insert, or delete tuples in the database.

The primary goal of object-relational mapping systems is to ease the job of programmers who build applications by providing them an object model while retaining the benefits of using a robust relational database underneath. As an added benefit, when operating on objects cached in memory, object-relational systems can provide significant performance gains over direct access to the underlying database.

Object-relational mapping systems also provide query languages that allow programmers to write queries directly on the object model; such queries are translated into SQL queries on the underlying relational database, and result objects are created from the SQL query results.

A fringe benefit of using an ORM is that any of a number of databases can be used to store data, with exactly the same high-level code. ORMs hide minor SQL differences between databases from the higher levels. Migration from one database to another is thus relatively straightforward when using an ORM, whereas SQL differences can make such migration significantly harder if an application uses SQL to communicate with the database.

On the negative side, object-relational mapping systems can suffer from significant performance inefficiencies for bulk database updates, as well as for complex queries that are written directly in the imperative language. It is possible to update the database directly, bypassing the object-relational mapping system, and to write complex queries directly in SQL in cases where such inefficiencies are discovered.

The benefits of object-relational models exceed the drawbacks for many applications, and object-relational mapping systems have seen widespread adoption in recent years. In particular, Hibernate has seen wide adoption with Java, while several ORMs including Django and SQLAlchemy are widely used with Python. More information on the Hibernate ORM system, which provides an object-relational mapping for Java, and the Django ORM system, which provides an object-relational mapping for Python, can be found in Section 9.6.2.

8.3 Textual Data

Textual data consists of unstructured text. The term **information retrieval** generally refers to the querying of unstructured textual data. In the traditional model used in the field of information retrieval, textual information is organized into *documents*. In a database, a text-valued attribute can be considered a document. In the context of the web, each web page can be considered to be a document.

8.3.1 Keyword Queries

Information retrieval systems support the ability to retrieve documents with some desired information. The desired documents are typically described by a set of **keywords**—for example, the keywords "database system" may be used to locate documents on database systems, and the keywords "stock" and "scandal" may be used to locate articles about stock-market scandals. Documents have associated with them a set of keywords; typically, all the words in the documents are considered keywords. A **keyword query** retrieves documents whose set of keywords contains all the keywords in the query.

In its simplest form, an information-retrieval system locates and returns all documents that contain all the keywords in the query. More-sophisticated systems estimate the relevance of documents to a query so that the documents can be shown in order of estimated relevance. They use information about keyword occurrences, as well as hyperlink information, to estimate relevance.

Keyword search was originally targeted at document repositories within organizations or domain-specific document repositories such as research publications. But information retrieval is also important for documents stored in a database.

Keyword-based information retrieval can be used not only for retrieving textual data, but also for retrieving other types of data, such as video and audio data, that have descriptive keywords associated with them. For instance, a video movie may have associated with it keywords such as its title, director, actors, and genre, while an image or video clip may have tags, which are keywords describing the image or video clip, associated with it.

Web search engines are, at core, information retrieval systems. They retrieve and store web pages by *crawling* the web. Users submit keyword queries, and the information retrieval part of the web search engine finds stored web pages containing the required keyword. Web search engines have today evolved beyond just retrieving web pages. Today, search engines aim to satisfy a user's information needs by judging what topic a query is about and displaying not only web pages judged as relevant but also other kinds of information about the topic. For example, given a query term "cricket", a search engine may display scores from ongoing or recent cricket matches, rather than just top-ranked documents related to cricket. As another example, in response to a query "New York", a search engine may show a map of New York and images of New York in addition to web pages related to New York.

8.3.2 Relevance Ranking

The set of all documents that contain the keywords in a query may be very large; in particular, there are billions of documents on the web, and most keyword queries on a web search engine find hundreds of thousands of documents containing some or all of the keywords. Not all the documents are equally relevant to a keyword query. Information-retrieval systems therefore estimate relevance of documents to a query and return only highly ranked documents as answers. Relevance ranking is not an exact science, but there are some well-accepted approaches.

8.3.2.1 Ranking Using TF-IDF

The word **term** refers to a keyword occurring in a document, or given as part of a query. The first question to address is, given a particular term t, how relevant is a particular document d to the term. One approach is to use the number of occurrences of the term in the document as a measure of its relevance, on the assumption that more relevant terms are likely to be mentioned many times in a document. Just counting the number of occurrences of a term is usually not a good indicator: first, the number of occurrences depends on the length of the document, and second, a document containing 10 occurrences of a term may not be 10 times as relevant as a document containing one occurrence.

One way of measuring $TF(d, t)$, the relevance of a term t to a document d, is:

$$TF(d, t) = \log\left(1 + \frac{n(d, t)}{n(d)}\right)$$

where $n(d)$ denotes the number of term occurrences in the document and $n(d, t)$ denotes the number of occurrences of term t in the document d. Observe that this metric takes the length of the document into account. The relevance grows with more occurrences of a term in the document, although it is not directly proportional to the number of occurrences.

Many systems refine the above metric by using other information. For instance, if the term occurs in the title, or the author list, or the abstract, the document would be considered more relevant to the term. Similarly, if the first occurrence of a term is late in the document, the document may be considered less relevant than if the first occurrence is early in the document. The above notions can be formalized by extensions of the formula we have shown for $TF(d, t)$. In the information retrieval community, the relevance of a document to a term is referred to as **term frequency** (**TF**), regardless of the exact formula used.

A query Q may contain multiple keywords. The relevance of a document to a query with two or more keywords is estimated by combining the relevance measures of the document for each keyword. A simple way of combining the measures is to add them up. However, not all terms used as keywords are equal. Suppose a query uses two terms, one of which occurs frequently, such as "database", and another that is less frequent, such as "Silberschatz". A document containing "Silberschatz" but not "database" should be ranked higher than a document containing the term "database" but not "Silberschatz".

To fix this problem, weights are assigned to terms using the **inverse document frequency** (**IDF**), defined as:

$$IDF(t) = \frac{1}{n(t)}$$

where $n(t)$ denotes the number of documents (among those indexed by the system) that contain the term t. The **relevance** of a document d to a set of terms Q is then defined as:

$$r(d, Q) = \sum_{t \in Q} TF(d, t) * IDF(t)$$

This measure can be further refined if the user is permitted to specify weights $w(t)$ for terms in the query, in which case the user-specified weights are also taken into account by multiplying $TF(t)$ by $w(t)$ in the preceding formula.

The above approach of using term frequency and inverse document frequency as a measure of the relevance of a document is called the **TF–IDF** approach.

Almost all text documents (in English) contain words such as "and," "or," "a," and so on, and hence these words are useless for querying purposes since their inverse document frequency is extremely low. Information-retrieval systems define a set of words, called **stop words**, containing 100 or so of the most common words, and ignore these words when indexing a document. Such words are not used as keywords, and they are discarded if present in the keywords supplied by the user.

Another factor taken into account when a query contains multiple terms is the **proximity** of the terms in the document. If the terms occur close to each other in the document, the document will be ranked higher than if they occur far apart. The formula for $r(d, Q)$ can be modified to take proximity of the terms into account.

Given a query Q, the job of an information-retrieval system is to return documents in descending order of their relevance to Q. Since there may be a very large number of documents that are relevant, information-retrieval systems typically return only the first few documents with the highest degree of estimated relevance and permit users to interactively request further documents.

8.3.2.2 Ranking Using Hyperlinks

Hyperlinks between documents can be used to decide on the overall importance of a document, independent of the keyword query; for example, documents linked from many other documents are considered more important.

The web search engine Google introduced **PageRank**, which is a measure of popularity of a page based on the popularity of pages that link to the page. Using the PageRank popularity measure to rank answers to a query gave results so much better than previously used ranking techniques that Google became the most widely used search engine in a rather short period of time.

Note that pages that are pointed to from many web pages are more likely to be visited, and thus should have a higher PageRank. Similarly, pages pointed to by web pages with a high PageRank will also have a higher probability of being visited, and thus should have a higher PageRank.

The PageRank of a document d is thus defined (circularly) based on the PageRank of other documents that link to document d. PageRank can be defined by a set of linear equations, as follows: First, web pages are given integer identifiers. The jump probability matrix T is defined with $T[i, j]$ set to the probability that a random walker who is following a link out of page i follows the link to page j. Assuming that each link

from i has an equal probability of being followed $T[i,j] = 1/N_i$, where N_i is the number of links out of page i. Then the PageRank $P[j]$ for each page j can be defined as:

$$P[j] = \delta/N + (1 - \delta) * \sum_{i=1}^{N} (T[i,j] * P[i])$$

where δ is a constant between 0 and 1, usually set to 0.15, and N is the number of pages.

The set of equations generated as above are usually solved by an iterative technique, starting with each $P[i]$ set to $1/N$. Each step of the iteration computes new values for each $P[i]$ using the P values from the previous iteration. Iteration stops when the maximum change in any $P[i]$ value in an iteration goes below some cutoff value.

Note that PageRank is a static measure, independent of the keyword query; given a keyword query, it is used in combination with TF-IDF scores of a document to judge its relevance of the document to the keyword query.

PageRank is not the only measure of the popularity of a site. Information about how often a site is visited is another useful measure of popularity. Further, search engines track what fraction of times users click on a page when it is returned as an answer. Keywords that occur in the anchor text associated with the hyperlink to a page are viewed as very important and are given a higher term frequency. These and a number of other factors are used to rank answers to a keyword query.

8.3.3 Measuring Retrieval Effectiveness

Ranking of results of a keyword query is not an exact science. Two metrics are used to measure how well an information-retrieval system is able to answer queries. The first, **precision**, measures what percentage of the retrieved documents are actually relevant to the query. The second, **recall**, measures what percentage of the documents relevant to the query were retrieved. Since search engines find a very large number of answers, and users typically stop after browsing some number (say, 10 or 20) of the answers, the precision and recall numbers are usually measured "@K", where K is the number of answers viewed. Thus, one can talk of precision@10 or recall@20.

8.3.4 Keyword Querying on Structured Data and Knowledge Graphs

Although querying on structured data are typically done using query languages such as SQL, users who are not familiar with the schema or the query language find it difficult to get information from such data. Based on the success of keyword querying in the context of information retrieval from the web, techniques have been developed to support keyword queries on structured and semi-structured data.

One approach is to represent the data using graphs, and then perform keyword queries on the graphs. For example, tuples can be treated as nodes in the graph, and foreign key and other connections between tuples can be treated as edges in the graph. Keyword search is then modeled as finding tuples containing the given keywords and finding connecting paths between them in the corresponding graph.

For example, a query "Zhang Katz" on a university database may find the *name* "Zhang" occurring in a *student* tuple, and the *name* "Katz" in an *instructor* tuple, a path through the *advisor* relation connecting the two tuples. Other paths, such as student "Zhang" taking a course taught by "Katz" may also be found in response to this query. Such queries may be used for ad hoc browsing and querying of data when the user does not know the exact schema and does not wish to take the effort to write an SQL query defining what she is searching for. Indeed it is unreasonable to expect lay users to write queries in a structured query language, whereas keyword querying is quite natural.

Since queries are not fully defined, they may have many different types of answers, which must be ranked. A number of techniques have been proposed to rank answers in such a setting, based on the lengths of connecting paths and on techniques for assigning directions and weights to edges. Techniques have also been proposed for assigning popularity ranks to tuples based on foreign key links. More information on keyword searching of structured data may be found in the bibliographic notes for this chapter, available online.

Further, knowledge graphs can be used along with textual information to answer queries. For example, knowledge graphs can be used to provide unique identifiers to entities, which are used to annotate mentions of the entities in textual documents. Now a particular mention of a person in a document may have the phrase "Stonebraker developed PostgreSQL"; from the context, the word Stonebraker may be inferred to be the database researcher "Michael Stonebraker" and annotated by linking the word Stonebraker to the entity "Michael Stonebraker". The knowledge graph may also record the fact that Stonebraker won the Turing award. A query asking for "turing award postgresql" can now be answered by using information from the document and the knowledge graph.[2]

Web search engines today use large knowledge graphs, in addition to crawled documents, to answer user queries.

8.4 Spatial Data

Spatial data support in database systems is important for efficiently storing, indexing, and querying of data on the basis of spatial locations.

Two types of spatial data are particularly important:

- **Geographic data** such as road maps, land-usage maps, topographic elevation maps, political maps showing boundaries, land-ownership maps, and so on. **Geographic information systems** are special-purpose database systems tailored for storing geographic data. Geographic data is based on a round-earth coordinate system, with latitude, longitude, and elevation.

[2]In this case the knowledge graph may already record that Stonebraker developed PostgreSQL, but there are many other pieces of information that may exist only in documents, and not in the knowledge graphs.

- **Geometric data**, which include spatial information about how objects—such as buildings, cars, or aircraft—are constructed. Geometric data is based on a two-dimensional or three-dimensional Euclidean space, with X, Y, and Z coordinates.

Geographic and geometric data types are supported by many database systems, such as Oracle Spatial and Graph, the PostGIS extension of PostgreSQL, SQL Server, and the IBM DB2 Spatial Extender.

In this section we describe the modeling and querying of spatial data; implementation techniques such as indexing and query processing techniques are covered in Chapter 14 and in Chapter 15.

The syntax for representing geographic and geometric data varies by database, although representations based on the **Open Geospatial Consortium** (**OGC**) standard are now increasingly supported. See the manuals of the database you use to learn more about the specific syntax supported by the database.

8.4.1 Representation of Geometric Information

Figure 8.5 illustrates how various geometric constructs can be represented in a database, in a normalized fashion. We stress here that geometric information can be represented in several different ways, only some of which we describe.

A *line segment* can be represented by the coordinates of its endpoints. For example, in a map database, the two coordinates of a point would be its latitude and longitude. A **polyline** (also called a **linestring**) consists of a connected sequence of line segments and can be represented by a list containing the coordinates of the endpoints of the segments, in sequence. We can approximately represent an arbitrary curve with polylines by partitioning the curve into a sequence of segments. This representation is useful for two-dimensional features such as roads; here, the width of the road is small enough relative to the size of the full map that it can be considered to be a line. Some systems also support *circular arcs* as primitives, allowing curves to be represented as sequences of arcs.

We can represent a *polygon* by listing its vertices in order, as in Figure 8.5.[3] The list of vertices specifies the boundary of a polygonal region. In an alternative representation, a polygon can be divided into a set of triangles, as shown in Figure 8.5. This process is called **triangulation**, and any polygon can be triangulated. The complex polygon can be given an identifier, and each of the triangles into which it is divided carries the identifier of the polygon. Circles and ellipses can be represented by corresponding types or approximated by polygons.

List-based representations of polylines or polygons are often convenient for query processing. Such non-first-normal-form representations are used when supported by the underlying database. So that we can use fixed-size tuples (in first normal form) for representing polylines, we can give the polyline or curve an identifier, and we can

[3]Some references use the term *closed polygon* to refer to what we call polygons and refer to polylines as open polygons.

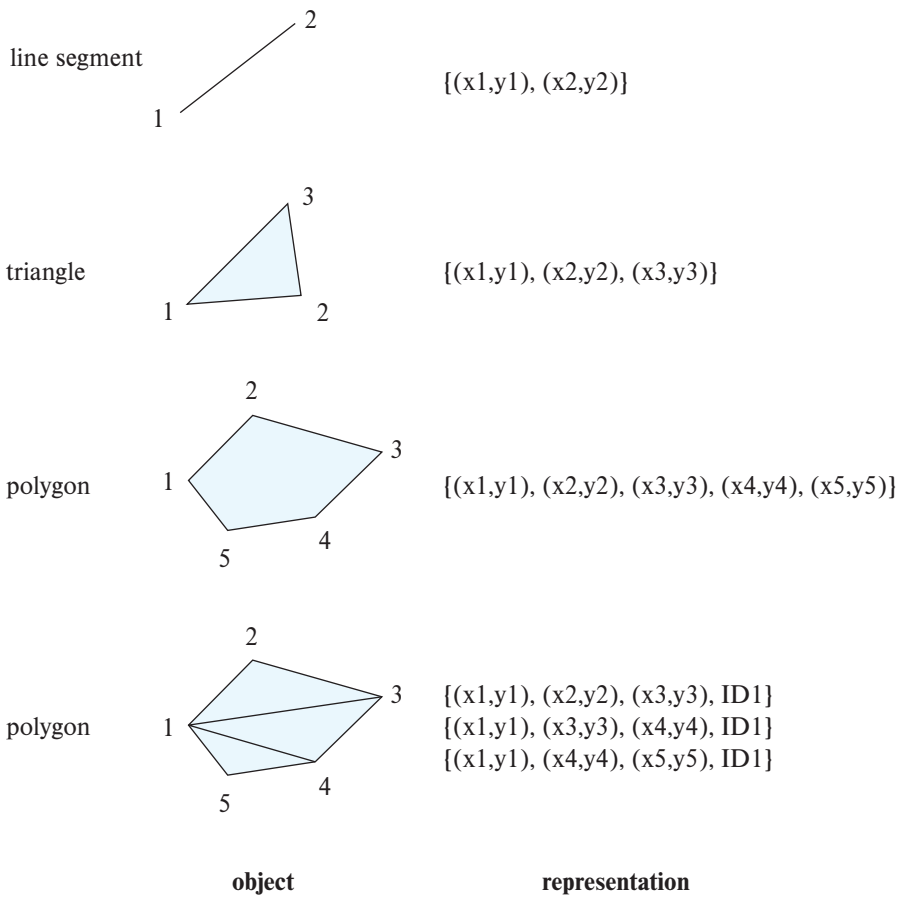


Figure 8.5 Representation of geometric constructs.

represent each segment as a separate tuple that also carries with it the identifier of the polyline or curve. Similarly, the triangulated representation of polygons allows a first normal form relational representation of polygons.

The representation of points and line segments in three-dimensional space is similar to their representation in two-dimensional space, the only difference being that points have an extra z component. Similarly, the representation of planar figures—such as triangles, rectangles, and other polygons—does not change much when we move to three dimensions. Tetrahedrons and cuboids can be represented in the same way as triangles and rectangles. We can represent arbitrary polyhedra by dividing them into tetrahedrons, just as we triangulate polygons. We can also represent them by listing their faces, each of which is itself a polygon, along with an indication of which side of the face is inside the polyhedron.

For example, SQL Server and PostGIS support the **geometry** and **geography** types, each of which has subtypes such as point, linestring, curve, polygon, as well as collections of these types called multipoint, multilinestring, multicurve and multipolygon. Textual representations of these types are defined by the OGC standards, and can be converted to internal representations using conversion functions. For example, LINESTRING(1 1, 2 3, 4 4) defines a line that connects points (1, 1), (2, 3) and (4, 4), while POLYGON((1 1, 2 3, 4 4, 1 1)) defines a triangle defined by these points. Functions *ST_GeometryFromText*() and *ST_GeographyFromText*() convert the textual representations to geometry and geography objects respectively. Operations on geometry and geography types that return objects of the same type include the *ST_Union*() and *ST_Intersection*() functions which compute the union and intersection of geometric objects such as linestrings and polygons. The function names as well as syntax differ by system; see the system manuals for details.

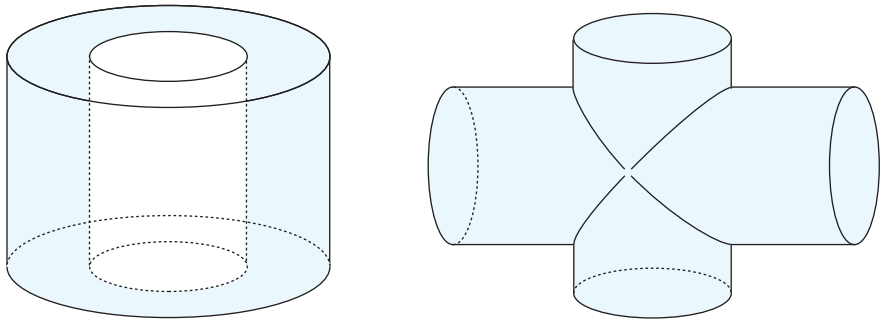
In the context of map data, the various line segments representing the roads are actually interconnected to form a graph. Such a **spatial network** or **spatial graph** has spatial locations for vertices of the graph, along with interconnection information between the vertices, which form the edges of the graph. The edges have a variety of associated information, such as distance, number of lanes, average speed at different times of the day, and so on.

8.4.2 Design Databases

Computer-aided-design (CAD) systems traditionally stored data in memory during editing or other processing and wrote the data back to a file at the end of a session of editing. The drawbacks of such a scheme include the cost (programming complexity, as well as time cost) of transforming data from one form to another and the need to read in an entire file even if only parts of it are required. For large designs, such as the design of a large-scale integrated circuit or the design of an entire airplane, it may be impossible to hold the complete design in memory. Designers of object-oriented databases were motivated in large part by the database requirements of CAD systems. Object-oriented databases represent components of the design as objects, and the connections between the objects indicate how the design is structured.

The objects stored in a design database are generally geometric objects. Simple two-dimensional geometric objects include points, lines, triangles, rectangles, and, in general, polygons. Complex two-dimensional objects can be formed from simple objects by means of union, intersection, and difference operations. Similarly, complex three-dimensional objects may be formed from simpler objects such as spheres, cylinders, and cuboids by union, intersection, and difference operations, as in Figure 8.6. Three-dimensional surfaces may also be represented by **wireframe models**, which essentially model the surface as a set of simpler objects, such as line segments, triangles, and rectangles.

Design databases also store nonspatial information about objects, such as the material from which the objects are constructed. We can usually model such information



(a) Difference of cylinders (b) Union of cylinders

Figure 8.6 Complex three-dimensional objects.

by standard data-modeling techniques. We concern ourselves here with only the spatial aspects.

Various spatial operations must be performed on a design. For instance, the designer may want to retrieve that part of the design that corresponds to a particular region of interest. Spatial-index structures, discussed in Section 14.10.1, are useful for such tasks. Spatial-index structures are multidimensional, dealing with two- and three-dimensional data, rather than dealing with just the simple one-dimensional ordering provided by the B^+-trees.

Spatial-integrity constraints, such as "two pipes should not be in the same location," are important in design databases to prevent interference errors. Such errors often occur if the design is performed manually and are detected only when a prototype is being constructed. As a result, these errors can be expensive to fix. Database support for spatial-integrity constraints helps people to avoid design errors, thereby keeping the design consistent. Implementing such integrity checks again depends on the availability of efficient multidimensional index structures.

8.4.3 Geographic Data

Geographic data are spatial in nature but differ from design data in certain ways. Maps and satellite images are typical examples of geographic data. Maps may provide not only location information—about boundaries, rivers, and roads, for example—but also much more detailed information associated with locations, such as elevation, soil type, land usage, and annual rainfall.

8.4.3.1 Applications of Geographic Data

Geographic databases have a variety of uses, including online map and navigation services, which are ubiquitous today. Other applications include distribution-network information for public-service utilities such as telephone, electric-power, and water-supply

systems, and land-usage information for ecologists and planners, land records to track land ownership, and many more.

Geographic databases for public-utility information have become very important as the network of buried cables and pipes has grown. Without detailed maps, work carried out by one utility may damage the structure of another utility, resulting in large-scale disruption of service. Geographic databases, coupled with accurate location-finding systems using GPS help avoid such problems.

8.4.3.2 Representation of Geographic Data

Geographic data can be categorized into two types:

- **Raster data**. Such data consist of bitmaps or pixel maps, in two or more dimensions. A typical example of a two-dimensional raster image is a satellite image of an area. In addition to the actual image, the data include the location of the image, specified, for example, by the latitude and longitude of its corners, and the resolution, specified either by the total number of pixels, or, more commonly in the context of geographic data, by the area covered by each pixel.

  Raster data are often represented as **tiles**, each covering a fixed-size area. A larger area can be displayed by displaying all the tiles that overlap with the area. To allow the display of data at different zoom levels, a separate set of tiles is created for each zoom level. Once the zoom level is set by the user interface (e.g., a web browser), tiles at the specified zoom level that overlap the area being displayed are retrieved and displayed.

  Raster data can be three-dimensional—for example, the temperature at different altitudes at different regions, again measured with the help of a satellite. Time could form another dimension—for example, the surface temperature measurements at different points in time.

- **Vector data**. Vector data are constructed from basic geometric objects, such as points, line segments, polylines, triangles, and other polygons in two dimensions, and cylinders, spheres, cuboids, and other polyhedrons in three dimensions. In the context of geographic data, points are usually represented by latitude and longitude, and where the height is relevant, additionally by elevation.

  Map data are often represented in vector format. Roads are often represented as polylines. Geographic features, such as large lakes, or even political features such as states and countries, are represented as complex polygons. Some features, such as rivers, may be represented either as complex curves or as complex polygons, depending on whether their width is relevant.

Geographic information related to regions, such as annual rainfall, can be represented as an array—that is, in raster form. For space efficiency, the array can be stored in a compressed form. In Section 24.4.1, we study an alternative representation of such arrays by a data structure called a *quadtree*.

As another alternative, we can represent region information in vector form, using polygons, where each polygon is a region within which the array value is the same. The vector representation is more compact than the raster representation in some applications. It is also more accurate for some tasks, such as depicting roads, where dividing the region into pixels (which may be fairly large) leads to a loss of precision in location information. However, the vector representation is unsuitable for applications where the data are intrinsically raster based, such as satellite images.

Topographical information, that is information about the elevation (height) of each point on a surface, can be represented in raster form. Alternatively, it can be represented in vector form by dividing the surface into polygons covering regions of (approximately) equal elevation, with a single elevation value associated with each polygon. As another alternative, the surface can be **triangulated** (i.e., divided into triangles), with each triangle represented by the latitude, longitude, and elevation of each of its corners. The latter representation, called the **triangulated irregular network** (**TIN**) representation, is a compact representation which is particularly useful for generating three-dimensional views of an area.

Geographic information systems usually contain both raster and vector data, and they can merge the two kinds of data when displaying results to users. For example, map applications usually contain both satellite images and vector data about roads, buildings, and other landmarks. A map display usually **overlays** different kinds of information; for example, road information can be overlaid on a background satellite image to create a hybrid display. In fact, a map typically consists of multiple layers, which are displayed in bottom-to-top order; data from higher layers appear on top of data from lower layers.

It is also interesting to note that even information that is actually stored in vector form may be converted to raster form before it is sent to a user interface such as a web browser. One reason is that even web browsers in which JavaScript has been disabled can then display map data; a second reason may be to prevent end users from extracting and using the vector data.

Map services such as Google Maps and Bing Maps provide APIs that allow users to create specialized map displays, containing application-specific data overlaid on top of standard map data. For example, a web site may show a map of an area with information about restaurants overlaid on the map. The overlays can be constructed dynamically, displaying only restaurants with a specific cuisine, for example, or allowing users to change the zoom level or pan the display.

8.4.4 Spatial Queries

There are a number of types of queries that involve spatial locations.

- **Region queries** deal with spatial regions. Such a query can ask for objects that lie partially or fully inside a specified region. A query to find all retail shops within the geographic boundaries of a given town is an example. PostGIS supports predicates between two geometry or geography objects such as *ST_Contains*(), *ST_Overlaps*(),

ST_Disjoint() and *ST_Touches*(). These can be used to find objects that are contained in, or intersect, or are disjoint from a region. SQL Server supports equivalent functions with slightly different names.
Suppose we have a *shop* relation, with an attribute *location* of type *point*, and a geography object of type *polygon*. Then the *ST_Contains*() function can be used to retrieve all shops whose location is contained in the given polygon.

- **Nearness queries** request objects that lie near a specified location. A query to find all restaurants that lie within a given distance of a given point is an example of a nearness query. The **nearest-neighbor query** requests the object that is nearest to a specified point. For example, we may want to find the nearest gasoline station. Note that this query does not have to specify a limit on the distance, and hence we can ask it even if we have no idea how far the nearest gasoline station lies.
  The PostGIS *ST_Distance*() function gives the minimum distance between two such objects, and can be used to find objects that are within a specified distance from a point or region. Nearest neighbors can be found by finding objects with minimum distance.
- **Spatial graph queries** request information based on spatial graphs such as road maps. For example, a query may ask for the shortest path between two locations via the road network, or via a train network, each of which can be represented as a spatial graph. Such queries are ubiquitous for navigation systems.

Queries that compute intersections of regions can be thought of as computing the **spatial join** of two spatial relations—for example, one representing rainfall and the other representing population density—with the location playing the role of join attribute. In general, given two relations, each containing spatial objects, the spatial join of the two relations generates either pairs of objects that intersect or the intersection regions of such pairs. Spatial predicates such as *ST_Contains*() or *ST_Overlaps*() can be used as join predicates when performing spatial joins.

In general, queries on spatial data may have a combination of spatial and nonspatial requirements. For instance, we may want to find the nearest restaurant that has vegetarian selections and that charges less than $10 for a meal.

8.5 Summary

- There are many application domains that need to store more complex data than simple tables with a fixed number of attributes.
- The SQL standard includes extensions of the SQL data-definition and query language to deal with new data types and with object orientation. These include support for collection-valued attributes, inheritance, and tuple references. Such extensions attempt to preserve the relational foundations—in particular, the declarative access to data—while extending the modeling power.

- Semi-structured data are characterized by complex data, whose schema changes often.
- A popular architecture for building information systems today is to create a web service that allows retrieval of data and to build application code that displays the data and allows user interaction.
- The relational data model has been extended in several ways to support the storage and data exchange needs of modern applications.
  - Some database systems allow each tuple to potentially have a different set of attributes.
  - Many data representations allow attributes to non-atomic values.
  - Many data representations allow attributes to be structured, directly modeling composite attributes in the E-R model.
- The **JavaScript Object Notation** (JSON) is a textual representation of complex data types which is widely used for transmitting data between applications and for storing complex data.
- XML representations provide flexibility in the set of attributes that a record contains as well as the types of these attributes.
- The **Resource Description Framework** (RDF) is a data representation standard based on the entity-relationship model. The RDF representation has a very natural graph interpretation. Entities and attribute values can be considered nodes, and attribute names and relationships can be considered edges between the nodes.
- SPARQL is a query language designed to query RDF data and is based on triple patterns.
- Object orientation provides inheritance with subtypes and subtables as well as object (tuple) references.
- The object-relational data model extends the relational data model by providing a richer type system, including collection types and object orientation.
- Object-relational database systems (i.e., database systems based on the object-relational model) provide a convenient migration path for users of relational databases who wish to use object-oriented features.
- Object-relational mapping systems provide an object view of data that are stored in a relational database. Objects are transient, and there is no notion of persistent object identity. Objects are created on demand from relational data, and updates to objects are implemented by updating the relational data. Object-relational mapping systems have been widely adopted, unlike the more limited adoption of persistent programming languages.

- Information-retrieval systems are used to store and query textual data such as documents. They use a simpler data model than do database systems but provide more powerful querying capabilities within the restricted model.
- Queries attempt to locate documents that are of interest by specifying, for example, sets of keywords. The query that a user has in mind usually cannot be stated precisely; hence, information-retrieval systems order answers on the basis of potential relevance.
- Relevance ranking makes use of several types of information, such as:
  - Term frequency: how important each term is to each document.
  - Inverse document frequency.
  - Popularity ranking.
- Spatial data management is important for many applications. Geometric and geographic data types are supported by many database systems, with subtypes including points, linestrings and polygons. Region queries, nearest neighbor queries, and spatial graph queries are among the commonly used types of spatial queries.

Review Terms

- Wide column
- Sparse column
- Key-value map
- Map
- Array database
- Tags
- Triples
- Resources
- Subject
- Predicate
- Object
- Knowledge graph
- Reification
- Quads
- Linked open data
- Object-relational data model
- Object-relational database system
- Object-relational mapping
- Object-oriented database system
- Path expression
- Keywords
- Keyword query
- Term
- Relevance
- TF-IDF
- Stop words
- Proximity
- PageRank
- Precision
- Recall
- Geographic data
- Geometric data
- Geographic information system
- Computer-aided-design (CAD)
- Polyline
- Linestring

- Triangulation
- Spatial network
- Spatial graph
- Raster data
- Tiles
- Vector data
- Topographical information
- Triangulated irregular network (TIN)
- Overlays
- Nearness queries
- Nearest-neighbor query
- Region queries
- Spatial graph queries
- Spatial join

Practice Exercises

8.1 Provide information about the student named Shankar in our sample university database, including information from the *student* tuple corresponding to Shankar, the *takes* tuples corresponding to Shankar and the *course* tuples corresponding to these *takes* tuples, in each of the following representations:

a. Using JSON, with an appropriate nested representation.

b. Using XML, with the same nested representation.

c. Using RDF triples.

d. As an RDF graph.

8.2 Consider the RDF representation of information from the university schema as shown in Figure 8.3. Write the following queries in SPARQL.

a. Find the titles of all courses taken by any student named Zhang.

b. Find titles of all courses such that a student named Zhang takes a section of the course that is taught by an instructor named Srinivasan.

c. Find the attribute names and values of all attributes of the instructor named Srinivasan, without enumerating the attribute names in your query.

8.3 A car-rental company maintains a database for all vehicles in its current fleet. For all vehicles, it includes the vehicle identification number, license number, manufacturer, model, date of purchase, and color. Special data are included for certain types of vehicles:

- Trucks: cargo capacity.
- Sports cars: horsepower, renter age requirement.
- Vans: number of passengers.
- Off-road vehicles: ground clearance, drivetrain (four- or two-wheel drive).

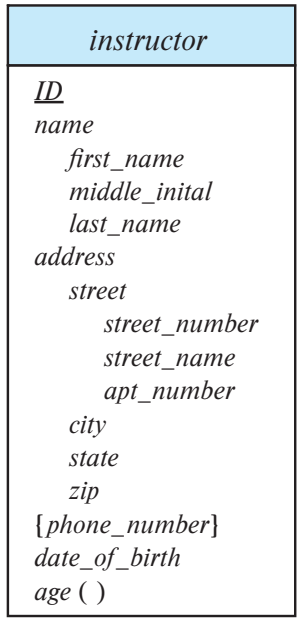


Figure 8.7 E-R diagram with composite, multivalued, and derived attributes.

Construct an SQL schema definition for this database. Use inheritance where appropriate.

8.4 Consider a database schema with a relation *Emp* whose attributes are as shown below, with types specified for multivalued attributes.

Emp = (ename, ChildrenSet **multiset***(Children), SkillSet* **multiset***(Skills))*
Children = (name, birthday)
Skills = (type, ExamSet **setof***(Exams))*
Exams = (year, city)

Define the above schema in SQL, using the SQL Server table type syntax from Section 8.2.1.1 to declare multiset attributes.

8.5 Consider the E-R diagram in Figure 8.7 showing entity set *instructor*. Give an SQL schema definition corresponding to the E-R diagram, treating *phone_number* as an array of 10 elements, using Oracle or PostgreSQL syntax.

8.6 Consider the relational schema shown in Figure 8.8.

a. Give a schema definition in SQL corresponding to the relational schema but using references to express foreign-key relationships.

b. Write each of the following queries on the schema, using SQL.

i. Find the company with the most employees.

employee (*person_name*, *street*, *city*)
works (*person_name*, *company_name*, *salary*)
company (*company_name*, *city*)
manages (*person_name*, *manager_name*)

Figure 8.8 Relational database for Exercise 8.6.

ii. Find the company with the smallest payroll.

iii. Find those companies whose employees earn a higher salary, on average, than the average salary at First Bank Corporation.

8.7 Compute the relevance (using appropriate definitions of term frequency and inverse document frequency) of each of the Practice Exercises in this chapter to the query "SQL relation".

8.8 Show how to represent the matrices used for computing PageRank as relations. Then write an SQL query that implements one iterative step of the iterative technique for finding PageRank; the entire algorithm can then be implemented as a loop containing the query.

8.9 Suppose the *student* relation has an attribute named *location* of type point, and the *classroom* relation has an attribute *location* of type polygon. Write the following queries in SQL using the PostGIS spatial functions and predicates that we saw earlier:

a. Find the names of all students whose location is within the classroom Packard 101.

b. Find all classrooms that are within 100 meters or Packard 101; assume all distances are represented in units of meters.

c. Find the ID and name of student who is geographically nearest to the student with ID 12345.

d. Find the ID and names of all pairs of students whose locations are less than 200 meters apart.

Exercises

8.10 Redesign the database of Exercise 8.4 into first normal form and fourth normal form. List any functional or multivalued dependencies that you assume. Also list all referential-integrity constraints that should be present in the first and fourth normal form schemas.

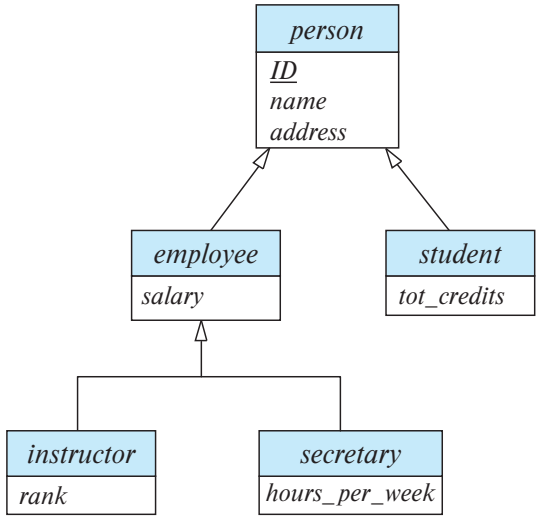


Figure 8.9 Specialization and generalization.

8.11 Consider the schemas for the table *people*, and the tables *students* and *teachers*, which were created under *people*, in Section 8.2.1.3. Give a relational schema in third normal form that represents the same information. Recall the constraints on subtables, and give all constraints that must be imposed on the relational schema so that every database instance of the relational schema can also be represented by an instance of the schema with inheritance.

8.12 Consider the E-R diagram in Figure 8.9, which contains specializations, using subtypes and subtables.

a. Give an SQL schema definition of the E-R diagram.

b. Give an SQL query to find the names of all people who are not secretaries.

c. Give an SQL query to print the names of people who are neither employees nor students.

d. Can you create a person who is an employee and a student with the schema you created? Explain how, or explain why it is not possible.

8.13 Suppose you wish to perform keyword querying on a set of tuples in a database, where each tuple has only a few attributes, each containing only a few words. Does the concept of term frequency make sense in this context? And that of inverse document frequency? Explain your answer. Also suggest how you can define the similarity of two tuples using TF-IDF concepts.

8.14 Web sites that want to get some publicity can join a web ring, where they create links to other sites in the ring in exchange for other sites in the ring creating links

to their site. What is the effect of such rings on popularity ranking techniques such as PageRank?

8.15 The Google search engine provides a feature whereby web sites can display advertisements supplied by Google. The advertisements supplied are based on the contents of the page. Suggest how Google might choose which advertisements to supply for a page, given the page contents.

Further Reading

A tutorial on JSON can be found at www.w3schools.com/js/js_json_intro.asp. More information about XML can be found in Chapter 30, available online. More information about RDF can be found at www.w3.org/RDF/. Apache Jena provides an RDF implementation, with support for SPARQL; a tutorial on SPARQL can be found at jena.apache.org/tutorials/sparql.html

POSTGRES ([Stonebraker and Rowe (1986)] and [Stonebraker (1986)]) was an early implementation of an object-relational system. Oracle provides a fairly complete implementation of the object-relational features of SQL, while PostgreSQL provides a smaller subset of those features. More information on support for these features may be found in their respective manuals.

[Salton (1989)] is an early textbook on information-retrieval systems, while [Manning et al. (2008)] is a modern textbook on the subject. Information about spatial database support in Oracle, PostgreSQL and SQL Server may be found in their respective manuals online.

Bibliography

[Manning et al. (2008)] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press (2008).

[Salton (1989)] G. Salton, *Automatic Text Processing*, Addison Wesley (1989).

[Stonebraker (1986)] M. Stonebraker, "Inclusion of New Types in Relational Database Systems", In *Proc. of the International Conf. on Data Engineering* (1986), pages 262–269.

[Stonebraker and Rowe (1986)] M. Stonebraker and L. Rowe, "The Design of POSTGRES", In *Proc. of the ACM SIGMOD Conf. on Management of Data* (1986), pages 340–355.

Credits

The photo of the sailboats in the beginning of the chapter is due to ©Pavel Nesvadba/Shutterstock

CHAPTER 9



Application Development

Practically all use of databases occurs from within application programs. Correspondingly, almost all user interaction with databases is indirect, via application programs. In this chapter, we study tools and technologies that are used to build applications, focusing on interactive applications that use databases to store and retrieve data.

A key requirement for any user-centric application is a good user interface. The two most common types of user interfaces today for database-backed applications are the web and mobile app interfaces.

In the initial part of this chapter, we provide an introduction to application programs and user interfaces (Section 9.1), and to web technologies (Section 9.2). We then discuss development of web applications using the widely used Java Servlets technology at the back end (Section 9.3), and using other frameworks (Section 9.4). Client-side code implemented using JavaScript or mobile app technologies is crucial for building responsive user interfaces, and we discuss some of these technologies (Section 9.5). We then provide an overview of web application architectures (Section 9.6) and cover performance issues in building large web applications (Section 9.7). Finally, we discuss issues in application security that are key to making applications resilient to attacks (Section 9.8), and encryption and its use in applications (Section 9.9).

9.1 Application Programs and User Interfaces

Although many people interact with databases, very few people use a query language to interact with a database system directly. The most common way in which users interact with databases is through an **application program** that provides a user interface at the front end and interfaces with a database at the back end. Such applications take input from users, typically through a forms-based interface, and either enter data into a database or extract information from a database based on the user input, and they then generate output, which is displayed to the user.

As an example of an application, consider a university registration system. Like other such applications, the registration system first requires you to identify and authenticate yourself, typically by a user name and password. The application then uses your

identity to extract information, such as your name and the courses for which you have registered, from the database and displays the information. The application provides a number of interfaces that let you register for courses and query other information, such as course and instructor information. Organizations use such applications to automate a variety of tasks, such as sales, purchases, accounting and payroll, human-resources management, and inventory management, among many others.

Application programs may be used even when it is not apparent that they are being used. For example, a news site may provide a page that is transparently customized to individual users, even if the user does not explicitly fill any forms when interacting with the site. To do so, it actually runs an application program that generates a customized page for each user; customization can, for example, be based on the history of articles browsed by the user.

A typical application program includes a front-end component, which deals with the user interface, a backend component, which communicates with a database, and a middle layer, which contains "business logic," that is, code that executes specific requests for information or updates, enforcing rules of business such as what actions should be carried out to execute a given task or who can carry out what task.

Applications such as airline reservations have been around since the 1960s. In the early days of computer applications, applications ran on large "mainframe" computers, and users interacted with the application through terminals, some of which even supported forms. The growth of personal computers resulted in the development of database applications with graphical user interfaces, or GUIs. These interfaces depended on code running on a personal computer that directly communicated with a shared database. Such an architecture was called a *client–server architecture*. There were two drawbacks to using such applications: first, user machines had direct access to databases, leading to security risks. Second, any change to the application or the database required all the copies of the application, located on individual computers, to be updated together.

Two approaches have evolved to avoid the above problems:

- Web browsers provide a *universal front end*, used by all kinds of information services. Browsers use a standardized syntax, the **HyperText Markup Language (HTML)** standard, which supports both formatted display of information and creation of forms-based interfaces. The HTML standard is independent of the operating system or browser, and pretty much every computer today has a web browser installed. Thus a web-based application can be accessed from any computer that is connected to the internet.

  Unlike client–server architectures, there is no need to install any application-specific software on client machines in order to use web-based applications.

  However, sophisticated user interfaces, supporting features well beyond what is possible using plain HTML, are now widely used, and are built with the scripting language JavaScript, which is supported by most web browsers. JavaScript programs, unlike programs written in C, can be run in a safe mode, guaranteeing

they cannot cause security problems. JavaScript programs are downloaded transparently to the browser and do not need any explicit software installation on the user's computer.

While the web browser provides the front end for user interaction, application programs constitute the back end. Typically, requests from a browser are sent to a web server, which in turn executes an application program to process the request. A variety of technologies are available for creating application programs that run at the back end, including Java servlets, Java Server Pages (JSP), Active Server Page (ASP), or scripting languages such as PHP and Python.

- Application programs are installed on individual devices, which are primarily mobile devices. They communicate with backend applications through an API and do not have direct access to the database. The back end application provides services, including user authentication, and ensures that users can only access services that they are authorized to access.

  This approach is widely used in mobile applications. One of the motivations for building such applications was to customize the display for the small screen of mobile devices. A second was to allow application code, which can be relatively large, to be downloaded or updated when the device is connected to a high-speed network, instead of downloading such code when a web page is accessed, perhaps over a lower bandwidth or more expensive mobile network.

With the increasing use of JavaScript code as part of web front ends, the difference between the two approaches above has today significantly decreased. The back end often provides an API that can be invoked from either mobile app or JavaScript code to carry out any required task at the back end. In fact, the same back end is often used to build multiple front ends, which could include web front ends with JavaScript, and multiple mobile platforms (primarily Android and iOS, today).

9.2 Web Fundamentals

In this section, we review some of the fundamental technology behind the World Wide Web, for readers who are not familiar with the technology underlying the web.

9.2.1 Uniform Resource Locators

A **uniform resource locator** (**URL**) is a globally unique name for each document that can be accessed on the web. An example of a URL is:

http://www.acm.org/sigmod

The first part of the URL indicates how the document is to be accessed: "http" indicates that the document is to be accessed by the **HyperText Transfer Protocol** (**HTTP**),

```
<html>
<body>
<table border>
<tr> <th>ID</th>       <th>Name</th>     <th>Department</th> </tr>
<tr> <td>00128</td> <td>Zhang</td>    <td>Comp. Sci.</td> </tr>
<tr> <td>12345</td> <td>Shankar</td> <td>Comp. Sci.</td> </tr>
<tr> <td>19991</td> <td>Brandt</td>    <td>History</td> </tr>
</table>
</body>
</html>
```

Figure 9.1 Tabular data in HTML format.

which is a protocol for transferring HTML documents; "https" would indicate that the secure version of the HTTP protocol must be used, and is the preferred mode today. The second part gives the name of a machine that has a web server. The rest of the URL is the path name of the file on the machine, or other unique identifier of a document within the machine.

A URL can contain the identifier of a program located on the web server machine, as well as arguments to be given to the program. An example of such a URL is

https://www.google.com/search?q=silberschatz

which says that the program search on the server www.google.com should be executed with the argument q=silberschatz. On receiving a request for such a URL, the web server executes the program, using the given arguments. The program returns an HTML document to the web server, which sends it back to the front end.

9.2.2 HyperText Markup Language

Figure 9.1 is an example of a table represented in the HTML format, while Figure 9.2 shows the displayed image generated by a browser from the HTML representation of the table. The HTML source shows a few of the HTML tags. Every HTML page should be enclosed in an html tag, while the body of the page is enclosed in a body tag. A table

ID	Name	Department
00128	Zhang	Comp. Sci.
12345	Shankar	Comp. Sci.
19991	Brandt	History

Figure 9.2 Display of HTML source from Figure 9.1.

```
<html>
<body>
<form action="PersonQuery" method=get>
Search for:
<select name="persontype">
      <option value="student" selected>Student </option>
      <option value="instructor"> Instructor </option>
</select> <br>
Name: <input type=text size=20 name="name">
<input type=submit value="submit">
</form>
</body>
</html>
```

Figure 9.3 An HTML form.

is specified by a table tag, which contains rows specified by a tr tag. The header row of the table has table cells specified by a th tag, while regular rows have table cells specified by a td tag. We do not go into more details about the tags here; see the bibliographical notes for references containing more detailed descriptions of HTML.

Figure 9.3 shows how to specify an HTML form that allows users to select the person type (student or instructor) from a menu and to input a number in a text box. Figure 9.4 shows how the above form is displayed in a web browser. Two methods of accepting input are illustrated in the form, but HTML also supports several other input methods. The action attribute of the form tag specifies that when the form is submitted (by clicking on the submit button), the form data should be sent to the URL PersonQuery (the URL is relative to that of the page). The web server is configured such that when this URL is accessed, a corresponding application program is invoked, with the user-provided values for the arguments persontype and name (specified in the select and input fields). The application program generates an HTML document, which is then sent back and displayed to the user; we shall see how to construct such programs later in this chapter.

HTTP defines two ways in which values entered by a user at the browser can be sent to the web server. The get method encodes the values as part of the URL. For example, if the Google search page used a form with an input parameter named q with the get

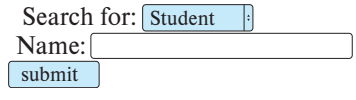


Figure 9.4 Display of HTML source from Figure 9.3.

method, and the user typed in the string "silberschatz" and submitted the form, the browser would request the following URL from the web server:

https://www.google.com/search?q=silberschatz

The post method would instead send a request for the URL https://www.google.com, and send the parameter values as part of the HTTP protocol exchange between the web server and the browser. The form in Figure 9.3 specifies that the form uses the get method.

Although HTML code can be created using a plain text editor, there are a number of editors that permit direct creation of HTML text by using a graphical interface. Such editors allow constructs such as forms, menus, and tables to be inserted into the HTML document from a menu of choices, instead of manually typing in the code to generate the constructs.

HTML supports *stylesheets*, which can alter the default definitions of how an HTML formatting construct is displayed, as well as other display attributes such as background color of the page. The *cascading stylesheet* (CSS) standard allows the same stylesheet to be used for multiple HTML documents, giving a distinctive but uniform look to all the pages on a web site. You can find more information on stylesheets online, for example at www.w3schools.com/css/.

The HTML5 standard, which was released in 2014, provides a wide variety of form input types, including the following:

- Date and time selection, using <input type="date" name="abc">, and <input type="time" name="xyz">. Browsers would typically display a graphical date or time picker for such an input field; the input value is saved in the form attributes abc and xyz. The optional attributes min and max can be used to specify minimum and maximum values that can be chosen.
- File selection, using <input type="file", name="xyz">, which allows a file to be chosen, and its name saved in the form attribute xyz.
- Input restrictions (constraints) on a variety of input types, including minimum, maximum, format matching a regular expression, and so on. For example, <input type="number" name="start" min="0" max="55" step="5" value="0"> allows the user to choose one of 0, 5, 10, 15, and so on till 55, with a default value of 0.

9.2.3 Web Servers and Sessions

A **web server** is a program running on the server machine that accepts requests from a web browser and sends back results in the form of HTML documents. The browser and web server communicate via HTTP. Web servers provide powerful features, beyond the simple transfer of documents. The most important feature is the ability to execute

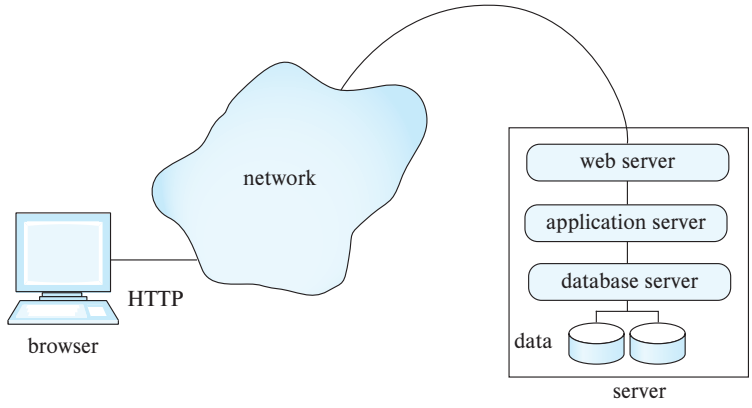


Figure 9.5 Three-layer web application architecture.

programs, with arguments supplied by the user, and to deliver the results back as an HTML document.

As a result, a web server can act as an intermediary to provide access to a variety of information services. A new service can be created by creating and installing an application program that provides the service. The **common gateway interface (CGI)** standard defines how the web server communicates with application programs. The application program typically communicates with a database server, through ODBC, JDBC, or other protocols, in order to get or store data.

Figure 9.5 shows a web application built using a three-layer architecture, with a web server, an application server, and a database server. Using multiple levels of servers increases system overhead; the CGI interface starts a new process to service each request, which results in even greater overhead.

Most web applications today therefore use a two-layer web application architecture, where the web and application servers are combined into a single server, as shown in Figure 9.6. We study systems based on the two-layer architecture in more detail in subsequent sections.

There is no continuous connection between the client and the web server; when a web server receives a request, a connection is temporarily created to send the request and receive the response from the web server. But the connection may then be closed, and the next request could come over a new connection. In contrast, when a user logs on to a computer, or connects to a database using ODBC or JDBC, a session is created, and session information is retained at the server and the client until the session is terminated—information such as the user-identifier of the user and session options that the user has set. One important reason that HTTP is **connectionless** is that most computers have limits on the number of simultaneous connections they can accommodate, and if a large number of sites on the web open connections to a single server, this limit would be exceeded, denying service to further users. With a connectionless protocol, the con-

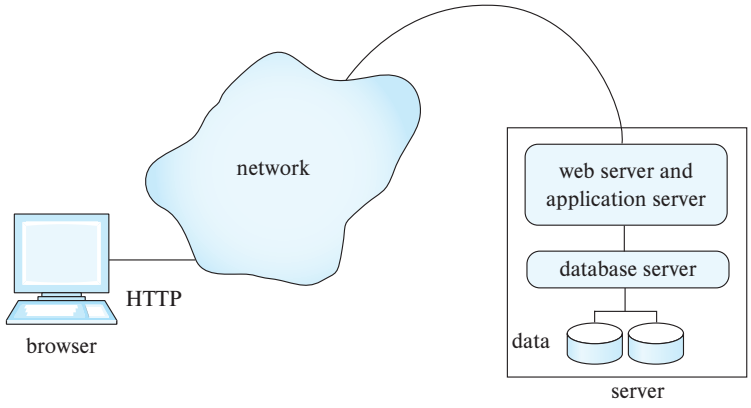


Figure 9.6 Two-layer web application architecture.

nection can be broken as soon as a request is satisfied, leaving connections available for other requests.[1]

Most web applications, however, need session information to allow meaningful user interaction. For instance, applications typically restrict access to information, and therefore need to authenticate users. Authentication should be done once per session, and further interactions in the session should not require reauthentication.

To implement sessions in spite of connections getting closed, extra information has to be stored at the client and returned with each request in a session; the server uses this information to identify that a request is part of a user session. Extra information about the session also has to be maintained at the server.

This extra information is usually maintained in the form of a **cookie** at the client; a cookie is simply a small piece of text containing identifying information and with an associated name. For example, google.com may set a cookie with the name prefs, which encodes preferences set by the user such as the preferred language and the number of answers displayed per page. On each search request, google.com can retrieve the cookie named prefs from the user's browser, and display results according to the specified preferences. A domain (web site) is permitted to retrieve only cookies that it has set, not cookies set by other domains, and cookie names can be reused across domains.

For the purpose of tracking a user session, an application may generate a session identifier (usually a random number not currently in use as a session identifier), and send a cookie named (for instance) sessionid containing the session identifier. The session identifier is also stored locally at the server. When a request comes in, the application server requests the cookie named sessionid from the client. If the client

[1]For performance reasons, connections may be kept open for a short while, to allow subsequent requests to reuse the connection. However, there is no guarantee that the connection will be kept open, and applications must be designed assuming the connection may be closed as soon as a request is serviced.

does not have the cookie stored, or returns a value that is not currently recorded as a valid session identifier at the server, the application concludes that the request is not part of a current session. If the cookie value matches a stored session identifier, the request is identified as part of an ongoing session.

If an application needs to identify users securely, it can set the cookie only after authenticating the user; for example a user may be authenticated only when a valid user name and password are submitted.[2]

For applications that do not require high security, such as publicly available news sites, cookies can be stored permanently at the browser and at the server; they identify the user on subsequent visits to the same site, without any identification information being typed in. For applications that require higher security, the server may invalidate (drop) the session after a time-out period, or when the user logs out. (Typically a user logs out by clicking on a logout button, which submits a logout form, whose action is to invalidate the current session.) Invalidating a session merely consists of dropping the session identifier from the list of active sessions at the application server.

9.3 Servlets

The Java **servlet** specification defines an application programming interface for communication between the web/application server and the application program. The `HttpServlet` class in Java implements the servlet API specification; servlet classes used to implement specific functions are defined as subclasses of this class.[3] Often the word *servlet* is used to refer to a Java program (and class) that implements the servlet interface. Figure 9.7 shows a servlet example; we explain it in detail shortly.

The code for a servlet is loaded into the web/application server when the server is started, or when the server receives a remote HTTP request to execute a particular servlet. The task of a servlet is to process such a request, which may involve accessing a database to retrieve necessary information, and dynamically generating an HTML page to be returned to the client browser.

9.3.1 A Servlet Example

Servlets are commonly used to generate dynamic responses to HTTP requests. They can access inputs provided through HTML forms, apply "business logic" to decide what

[2]The user identifier could be stored at the client end, in a cookie named, for example, `userid`. Such cookies can be used for low-security applications, such as free web sites identifying their users. However, for applications that require a higher level of security, such a mechanism creates a security risk: The value of a cookie can be changed at the browser by a malicious user, who can then masquerade as a different user. Setting a cookie (named `sessionid`, for example) to a randomly generated session identifier (from a large space of numbers) makes it highly improbable that a user can masquerade as (i.e., pretend to be) another user. A sequentially generated session identifier, on the other hand, would be susceptible to masquerading.

[3]The servlet interface can also support non-HTTP requests, although our example uses only HTTP.

response to provide, and then generate HTML output to be sent back to the browser. Servlet code is executed on a web or application server.

Figure 9.7 shows an example of servlet code to implement the form in Figure 9.3. The servlet is called PersonQueryServlet, while the form specifies that "action="PersonQuery"." The web/application server must be told that this servlet is to be used to handle requests for PersonQuery, which is done by using the anno-

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

@WebServlet("PersonQuery")
public class PersonQueryServlet extends HttpServlet {
    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException
    {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        ... check if user is logged in ...
        out.println("<HEAD><TITLE> Query Result</TITLE></HEAD>");
        out.println("<BODY>");

        String persontype = request.getParameter("persontype");
        String name = request.getParameter("name");
        if(persontype.equals("student")) {
            ... code to find students with the specified name ...
            ... using JDBC to communicate with the database ..
            ... Assume ResultSet rs has been retrieved, and
            ... contains attributes ID, name, and department name
            String headers = new String[]{"ID", "Name", "Department Name"};
            Util::resultSetToHTML(rs, headers, out);
        }
        else {
            ... as above, but for instructors ...
        }
        out.println("</BODY>");
        out.close();
    }
}
```

Figure 9.7 Example of servlet code.

tation @WebServlet("PersonQuery") shown in the code. The form specifies that the HTTP get mechanism is used for transmitting parameters. So the doGet() method of the servlet, as defined in the code, is invoked.

Each servlet request results in a new thread within which the call is executed, so multiple requests can be handled in parallel. Any values from the form menus and input fields on the web page, as well as cookies, pass through an object of the HttpServletRequest class that is created for the request, and the reply to the request passes through an object of the class HttpServletResponse.

The doGet() method in the example extracts values of the parameters persontype and name by using request.getParameter(), and uses these values to run a query against a database. The code used to access the database and to get attribute values from the query result is not shown; refer to Section 5.1.1.5 for details of how to use JDBC to access a database. We assume that the result of the query in the form of a JDBC ResultSet is available in the variable resultset.

The servlet code returns the results of the query to the requester by outputting them to the HttpServletResponse object response. Outputting the results to response is implemented by first getting a PrintWriter object out from response, and then printing the query result in HTML format to out. In our example, the query result is printed by calling the function Util::resultSetToHTML(resultset, header, out), which is shown in Figure 9.8. The function uses JDBC metadata function on the ResultSet rs to figure out how many columns need to be printed. An array of column headers is passed to this function to be printed out; the column names could have been obtained using JDBC metadata, but the database column names may not be appropriate for display to a user, so we provide meaningful column names to the function.

9.3.2 Servlet Sessions

Recall that the interaction between a browser and a web/application server is stateless. That is, each time the browser makes a request to the server, the browser needs to connect to the server, request some information, then disconnect from the server. Cookies can be used to recognize that a request is from the same browser session as an earlier request. However, cookies form a low-level mechanism, and programmers require a better abstraction to deal with sessions.

The servlet API provides a method of tracking a session and storing information pertaining to it. Invocation of the method getSession(false) of the class HttpServletRequest retrieves the HttpSession object corresponding to the browser that sent the request. An argument value of true would have specified that a new session object must be created if the request is a new request.

When the getSession() method is invoked, the server first asks the client to return a cookie with a specified name. If the client does not have a cookie of that name, or returns a value that does not match any ongoing session, then the request is not part of an ongoing session. In this case, getSession() would return a null value, and the servlet could direct the user to a login page.

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class Util {
    public static void resultSetToHTML(ResultSet rs,
                                       String headers[], PrintWriter out) {
        ResultSetMetaData rsmd = rs.getMetaData();
        int numCols = rsmd.getColumnCount();
        out.println("<table border=1>");
        out.println("<tr>");
        for (int i=0; i < numCols; i++)
            out.println("<th>"+ headers[i] + </th>
        out.println("</tr>");
        while (rs.next()) {
            out.println("<tr>");
            for (int i=0; i < numCols; i++)
                out.println("<td>"+ rs.getString(i) + </td>
            out.println("</tr>");
        }
    }
}
```

Figure 9.8 Utility function to output ResultSet as a table.

The login page could allow the user to provide a user name and password. The servlet corresponding to the login page could verify that the password matches the user; for example, by using the user name to retrieve the password from the database and checking if the password entered matches the stored password.[4]

If the user is properly authenticated, the login servlet would execute getSession(true), which would return a new session object. To create a new session, the server would internally carry out the following tasks: set a cookie (called, for example, sessionId) with a session identifier as its associated value at the client browser, create a new session object, and associate the session identifier value with the session object.

[4]It is a bad idea to store unencrypted passwords in the database, since anyone with access to the database contents, such as a system administrator or a hacker, could steal the password. Instead, a hashing function is applied to the password, and the result is stored in the database; even if someone sees the hashing result stored in the database, it is very hard to infer what was the original password. The same hashing function is applied to the user-supplied password, and the result is compared with the stored hashing result. Further, to ensure that even if two users use the same password the hash values are different, the password system typically stores a different random string (called the *salt*) for each user, and it appends the random string to the password before computing the hash value. Thus, the password relation would have the schema *user_password*(*user, salt, passwordhash*), where *passwordhash* is generated by *hash*(*append*(*password*,*salt*)). Encryption is described in more detail in Section 9.9.1.

The servlet code can also store and look up (attribute-name, value) pairs in the HttpSession object, to maintain state across multiple requests within a session. For example, after the user is authenticated and the session object has been created, the login servlet could store the user-id of the user as a session parameter by executing the method

```
session.setAttribute("userid", userid)
```

on the session object returned by getSession(); the Java variable userid is assumed to contain the user identifier.

If the request was part of an ongoing session, the browser would have returned the cookie value, and the corresponding session object would be returned by getSession(). The servlet could then retrieve session parameters such as user-id from the session object by executing the method

```
session.getAttribute("userid")
```

on the session object returned above. If the attribute userid is not set, the function would return a null value, which would indicate that the client user has not been authenticated.

Consider the line in the servlet code in Figure 9.7 that says "... check if user is logged in...". The following code implements this check; if the user is not logged in, it sends an error message, and after a gap of 5 seconds, redirects the user to the login page.

```
Session session = request.getSession(false);
if (session == null || session.getAttribute(userid) == null) {
      out.println("You are not logged in.");
      response.setHeader("Refresh", "5;url=login.html");
      return();
}
```

9.3.3 Servlet Life Cycle

The life cycle of a servlet is controlled by the web/application server in which the servlet has been deployed. When there is a client request for a specific servlet, the server first checks if an instance of the servlet exists or not. If not, the server loads the servlet class into the Java virtual machine (JVM) and creates an instance of the servlet class. In addition, the server calls the init() method to initialize the servlet instance. Notice that each servlet instance is initialized only once when it is loaded.

After making sure the servlet instance does exist, the server invokes the service method of the servlet, with a request object and a response object as parameters. By default, the server creates a new thread to execute the service method; thus, multiple

requests on a servlet can execute in parallel, without having to wait for earlier requests to complete execution. The service method calls doGet or doPost as appropriate.

When no longer required, a servlet can be shut down by calling the destroy() method. The server can be set up to shut down a servlet automatically if no requests have been made on a servlet within a time-out period; the time-out period is a server parameter that can be set as appropriate for the application.

9.3.4 Application Servers

Many application servers provide built-in support for servlets. One of the most popular is the Tomcat Server from the Apache Jakarta Project. Other application servers that support servlets include Glassfish, JBoss, BEA Weblogic Application Server, Oracle Application Server, and IBM's WebSphere Application Server.

The best way to develop servlet applications is by using an IDE such as Eclipse or NetBeans, which come with Tomcat or Glassfish servers built in.

Application servers usually provide a variety of useful services, in addition to basic servlet support. They allow applications to be deployed or stopped, and they provide functionality to monitor the status of the application server, including performance statistics. Many application servers also support the Java 2 Enterprise Edition (J2EE) platform, which provides support and APIs for a variety of tasks, such as for handling objects, and parallel processing across multiple application servers.

9.4 Alternative Server-Side Frameworks

There are several alternatives to Java Servlets for processing requests at the application server, including scripting languages and web application frameworks developed for languages such as Python.

9.4.1 Server-Side Scripting

Writing even a simple web application in a programming language such as Java or C is a time-consuming task that requires many lines of code and programmers who are familiar with the intricacies of the language. An alternative approach, that of **server-side scripting**, provides a much easier method for creating many applications. Scripting languages provide constructs that can be embedded within HTML documents.

In server-side scripting, before delivering a web page, the server executes the scripts embedded within the HTML contents of the page. Each piece of script, when executed, can generate text that is added to the page (or may even delete content from the page). The source code of the scripts is removed from the page, so the client may not even be aware that the page originally had any code in it. The executed script may also contain SQL code that is executed against a database. Many of these languages come with libraries and tools that together constitute a framework for web application development.

```
<html>
<head> <title> Hello </title> </head>
<body>
  < % if (request.getParameter("name") == null)
    { out.println("Hello World"); }
    else { out.println("Hello, " + request.getParameter("name")); }
  %>
</body>
</html>
```

Figure 9.9 A JSP page with embedded Java code.

Some of the widely used scripting frameworks include Java Server Pages (JSP), ASP.NET from Microsoft, PHP, and Ruby on Rails. These frameworks allow code written in languages such as Java, C#, VBScript, and Ruby to be embedded into or invoked from HTML pages. For instance, JSP allows Java code to be embedded in HTML pages, while Microsoft's ASP.NET and ASP support embedded C# and VBScript.

9.4.1.1 Java Server Pages

Next we briefly describe **Java Server Pages** (**JSP**), a scripting language that allows HTML programmers to mix static HTML with dynamically generated HTML. The motivation is that, for many dynamic web pages, most of their content is still static (i.e., the same content is present whenever the page is generated). The dynamic content of the web pages (which are generated, for example, on the basis of form parameters) is often a small part of the page. Creating such pages by writing servlet code results in a large amount of HTML being coded as Java strings. JSP instead allows Java code to be embedded in static HTML; the embedded Java code generates the dynamic part of the page. JSP scripts are actually translated into servlet code that is then compiled, but the application programmer is saved the trouble of writing much of the Java code to create the servlet.

Figure 9.9 shows the source text of a JSP page that includes embedded Java code. The Java code in the script is distinguished from the surrounding HTML code by being enclosed in <% … %>. The code uses `request.getParameter()` to get the value of the attribute `name`.

When a JSP page is requested by a browser, the application server generates HTML output from the page, which is sent to the browser. The HTML part of the JSP page is output as is.[5] Wherever Java code is embedded within <% …%>, the code is replaced in the HTML output by the text it prints to the object `out`. In the JSP code in Figure 9.9,

[5] JSP allows a more complex embedding, where HTML code is within a Java if-else statement, and gets output conditionally depending on whether the if condition evaluates to true or not. We omit details here.

if no value was entered for the form parameter name, the script prints "Hello World"; if a value was entered, the script prints "Hello" followed by the name.

A more realistic example may perform more complex actions, such as looking up values from a database using JDBC.

JSP also supports the concept of a *tag library*, which allows the use of tags that look much like HTML tags but are interpreted at the server and are replaced by appropriately generated HTML code. JSP provides a standard set of tags that define variables and control flow (iterators, if-then-else), along with an expression language based on JavaScript (but interpreted at the server). The set of tags is extensible, and a number of tag libraries have been implemented. For example, there is a tag library that supports paginated display of large data sets and a library that simplifies display and parsing of dates and times.

9.4.1.2 PHP

PHP is a scripting language that is widely used for server-side scripting. PHP code can be intermixed with HTML in a manner similar to JSP. The characters "<?php" indicate the start of PHP code, while the characters "?>" indicate the end of PHP code. The following code performs the same actions as the JSP code in Figure 9.9.

```
<html>
<head> <title> Hello </title> </head>
<body>
  <?php if (!isset($_REQUEST['name']))
    { echo 'Hello World'; }
    else { echo 'Hello, ' . $_REQUEST['name']; }
  ?>
</body>
</html>
```

The array $_REQUEST contains the request parameters. Note that the array is indexed by the parameter name; in PHP arrays can be indexed by arbitrary strings, not just numbers. The function isset checks if the element of the array has been initialized. The echo function prints its argument to the output HTML. The operator "." between two strings concatenates the strings.

A suitably configured web server would interpret any file whose name ends in ".php" to be a PHP file. If the file is requested, the web server processes it in a manner similar to how JSP files are processed and returns the generated HTML to the browser.

A number of libraries are available for the PHP language, including libraries for database access using ODBC (similar to JDBC in Java).

9.4.2 Web Application Frameworks

Web application development frameworks ease the task of constructing web applications by providing features such as these:

- A library of functions to support HTML and HTTP features, including sessions.
- A template scripting system.
- A controller that maps user interaction events such as form submits to appropriate functions that handle the event. The controller also manages authentication and sessions. Some frameworks also provide tools for managing authorizations.
- A (relatively) declarative way of specifying a form with validation constraints on user inputs, from which the system generates HTML and Javascript/Ajax code to implement the form.
- An object-oriented model with an object-relational mapping to store data in a relational database (described in Section 9.6.2).

Thus, these frameworks provide a variety of features that are required to build web applications in an integrated manner. By generating forms from declarative specifications and managing data access transparently, the frameworks minimize the amount of coding that a web application programmer has to carry out.

There are a large number of such frameworks, based on different languages. Some of the more widely used frameworks include the Django framework for the Python language, Ruby on Rails, which supports the Rails framework on the Ruby programming language, Apache Struts, Swing, Tapestry, and WebObjects, all based on Java/JSP. Many of these frameworks also make it easy to create simple **CRUD** web interfaces; that is, interfaces that support create, read, update and delete of objects/tuples by generating code from an object model or a database. Such tools are particularly useful to get simple applications running quickly, and the generated code can be edited to build more sophisticated web interfaces.

9.4.3 The Django Framework

The Django framework for Python is a widely used web application framework. We illustrate a few features of the framework through examples.

Views in Django are functions that are equivalent to servlets in Java. Django requires a mapping, typically specified in a file `urls.py`, which maps URLs to Django views. When the Django application server receives an HTTP request, it uses the URL mapping to decide which view function to invoke.

Figure 9.10 shows sample code implementing the person query task that we earlier implemented using Java servlets. The code shows a view called `person_query_view`. We assume that the `PersonQuery` URL is mapped to the view `person_query_view`, and is invoked from the HTML form shown earlier in Figure 9.3.

We also assume that the root of the application is mapped to a `login_view`. We have not shown the code for `login_view`, but we assume it displays a login form, and on submit it invokes the `authenticate` view. We have not shown the `authenticate` view,

either, but we assume that it checks the login name and password. If the password is validated, the authenticate view redirects to a person_query_form, which displays the HTML code that we saw earlier in Figure 9.3; if password validation fails, it redirects to the login_view.

Returning to Figure 9.10, the view person_query_view() first checks if the user is logged in by checking the session variable username. If the session variable is not set, the browser is redirected to the login screen. Otherwise, the requested user informa-

```
from django.http import HttpResponse
from django.db import connection

def result_set_to_html(headers, cursor):
    html = "<table border=1>"
    html += "<tr>"
    for header in headers:
        html += "<th>" + header + "</th>"
    html += "</tr>"
    for row in cursor.fetchall():
        html += "<tr>"
        for col in row:
            html += "<td>" + col + "</td>"
        html += "</tr>"
    html += "</table>"
    return html

def person_query_view(request):
    if "username" not in request.session:
        return login_view(request)
    persontype = request.GET.get("persontype")
    personname = request.GET.get("personname")
    if persontype == "student":
        query_tmpl = "select id, name, dept_name from student where name=%s"
    else:
        query_tmpl = "select id, name, dept_name from instructor where name=%s"
    with connection.cursor() as cursor:
        cursor.execute(query_tmpl, [personname])
        headers = ["ID", "Name", "Department Name"]
        return HttpResponse(result_set_to_html(headers, cursor))
```

Figure 9.10 The person query application in Django.

tion is fetched by connecting to the database; connection details for the database are specified in a Django configuration file settings.py and are omitted in our description. A cursor (similar to a JDBC statement) is opened on the connection, and the query is executed using the cursor. Note that the first argument of cursor.execute is the query, with parameters marked by "%s", and the second argument is a list of values for the parameters. The result of the database query is then displayed by calling a function result_set_to_html(), which iterates over the result set fetched from the database and outputs the results in HTML format to a string; the string is then returned as an HttpResponse.

Django provides support for a number of other features, such as creating HTML forms and validating data entered in the forms, annotations to simplify checking of authentication, and templates for creating HTML pages, which are somewhat similar to JSP pages. Django also supports an object-relation mapping system, which we describe in Section 9.6.2.2.

9.5 Client-Side Code and Web Services

The two most widely used classes of user interfaces today are the web interfaces and mobile application interfaces.

While early generation web browsers only displayed HTML code, the need was soon felt to allow code to run on the browsers. **Client-side scripting languages** are languages designed to be executed on the client's web browser. The primary motivation for such scripting languages is flexible interaction with the user, providing features beyond the limited interaction power provided by HTML and HTML forms. Further, executing programs at the client site speeds up interaction greatly compared to every interaction being sent to a server site for processing.

The **JavaScript** language is by far the most widely used client-side scripting language. The current generation of web interfaces uses the JavaScript scripting language extensively to construct sophisticated user interfaces.

Any client-side interface needs to store and retrieve data from the back end. Directly accessing a database is not a good idea, since it not only exposes low-level details, but it also exposes the database to attacks. Instead, back ends provide access to store and retrieve data through web services. We discuss web services in Section 9.5.2.

Mobile applications are very widely used, and user interfaces for mobile devices are very important today. Although we do not cover mobile application development in this book, we offer pointers to some mobile application development frameworks in Section 9.5.4.

9.5.1 JavaScript

JavaScript is used for a variety of tasks, including validation, flexible user interfaces, and interaction with web services, which we now describe.

9.5.1.1 Input Validation

Functions written in JavaScript can be used to perform error checks (validation) on user input, such as a date string being properly formatted, or a value entered (such as age) being in an appropriate range. These checks are carried out on the browser as data are entered even before the data are sent to the web server.

With HTML5, many validation constraints can be specified as part of the input tag. For example, the following HTML code:

```
<input type="number" name="credits" size="2" min="1" max="15">
```

ensures that the input for the parameter "credits" is a number between 1 and 15. More complex validations that cannot be performed using HTML5 features are best done using JavaScript.

Figure 9.11 shows an example of a form with a JavaScript function used to validate a form input. The function is declared in the head section of the HTML document. The form accepts a start and an end date. The validation function ensures that the start date

```
<html>
<head>
<script type="text/javascript">
    function validate() {
        var startdate = new Date (document.getElementById("start").value);
        var enddate = new Date (document.getElementById("end").value);
        if(startdate > enddate) {
            alert("Start date is > end date");
            return false;
        }
  }
</script>
</head>

<body>
<form action="submitDates" onsubmit="return validate()">
    Start Date: <input type="date" id="start"><br />
    End Date : <input type="date" id="end"><br />
    <input type="submit" value="Submit">
</form>
</body>
</html>
```

Figure 9.11 Example of JavaScript used to validate form input.

is not greater than the end date. The form tag specifies that the validation function is to be invoked when the form is submitted. If the validation fails, an alert box is shown to the user, and if it succeeds, the form is submitted to the server.

9.5.1.2 Responsive User Interfaces

The most important benefit of JavaScript is the ability to create highly responsive user interfaces within a browser using JavaScript. The key to building such a user interface is the ability to dynamically modify the HTML code being displayed by using JavaScript. The browser parses HTML code into an in-memory tree structure defined by a standard called the **Document Object Model** (**DOM**). JavaScript code can modify the tree structure to carry out certain operations. For example, suppose a user needs to enter a number of rows of data, for example multiple items in a single bill. A table containing text boxes and other form input methods can be used to gather user input. The table may have a default size, but if more rows are needed, the user may click on a button labeled (for example) "Add Item." This button can be set up to invoke a JavaScript function that modifies the DOM tree by adding an extra row in the table.

Although the JavaScript language has been standardized, there are differences between browsers, particularly in the details of the DOM model. As a result, JavaScript code that works on one browser may not work on another. To avoid such problems, it is best to use a JavaScript library, such as the JQuery library, which allows code to be written in a browser-independent way. Internally, the functions in the library can find out which browser is in use and send appropriately generated JavaScript to the browser.

JavaScript libraries such as JQuery provide a number of UI elements, such as menus, tabs, widgets such as sliders, and features such as autocomplete, that can be created and executed using library functions.

The HTML5 standard supports a number of features for rich user interaction, including drag-and-drop, geolocation (which allows the user's location to be provided to the application with user permission), allowing customization of the data/interface based on location. HTML5 also supports Server-Side Events (SSE), which allows a backend to notify the front end when some event occurs.

9.5.1.3 Interfacing with Web Services

Today, JavaScript is widely used to create dynamic web pages, using several technologies that are collectively called **Ajax**. Programs written in JavaScript can communicate with the web server asynchronously (that is, in the background, without blocking user interaction with the web browser), and can fetch data and display it. The *JavaScript Object Notation*, or JSON, representation described in Section 8.1.2 is the most widely used data format for transferring data, although other formats such as XML are also used.

The role of the code for the above tasks, which runs at the application server, is to send data to the JavaScript code, which then renders the data on the browser.

Such backend services, which serve the role of functions which can be invoked to fetch required data, are known as *web services*. Such services can be implemented using Java Servlets, Python, or any of a number of other language frameworks.

As an example of the use of Ajax, consider the autocomplete feature implemented by many web applications. As the user types a value in a text box, the system suggests completions for the value being typed. Such autocomplete is very useful for helping a user choose a value from a large number of values where a drop-down list would not be feasible. Libraries such as jQuery provide support for autocomplete by associating a function with a text box; the function takes partial input in the box, connected to a web back end to get possible completions, and displays them as suggestions for the autocomplete.

The JavaScript code shown in Figure 9.12 uses the jQuery library to implement autocomplete and the DataTables plug-in for the jQuery library to provide a tabular display of data. The HTML code has a text input box for name, which has an id attribute set to name. The script associates an autocomplete function from the jQuery library with the text box by using $("#name") syntax of jQuery to locate the DOM node for text box with id "name", and then associating the autocomplete function with the node. The attribute source passed to the function identifies the web service that must be invoked to get values for the autocomplete functionality. We assume that a servlet /autocomplete_name has been defined, which accepts a parameter term containing the letters typed so far by the user, even as they are being typed. The servlet should return a JSON array of names of students/instructors that match the letters in the term parameter.

The JavaScript code also illustrates how data can be retrieved from a web service and then displayed. Our sample code uses the DataTables jQuery plug-in; there are a number of other alternative libraries for displaying tabular data. We assume that the person_query_ajax Servlet, which is not shown, returns the ID, name, and department name of students or instructors with a given name, as we saw earlier in Figure 9.7, but encoded in JSON as an object with attribute data containing an array of rows; each row is a JSON object with attributes id, name, and dept_name.

The line starting with myTable shows how the jQuery plug-in DataTable is associated with the HTML table shown later in the figure, whose identifier is personTable. When the button "Show details" is clicked, the function loadTableAsync() is invoked. This function first creates a URL string url that is used to invoke person_query_ajax with values for person type and name. The function ajax.url(url).load() invoked on myTable fills the rows of the table using the JSON data fetched from the web service whose URL we created above. This happens asynchronously; that is, the function returns immediately, but when the data have been fetched, the table rows are filled with the returned data.

Figure 9.13 shows a screenshot of a browser displaying the result of the code in Figure 9.12.

As another example of the use of Ajax, consider a web site with a form that allows you to select a country, and once a country has been selected, you are allowed to select

```
<html> <head>
<script src="https://code.jquery.com/jquery-3.3.1.js"> </script>
<script src="https://cdn.datatables.net/1.10.19/js/jquery.dataTables.min.js"></script>
<script src="https://code.jquery.com/ui/1.12.1/jquery-ui.min.js"></script>
<script src="https://cdn.datatables.net/1.10.19/js/jquery.dataTables.min.js"></script>
<link rel="stylesheet"
    href="https://code.jquery.com/ui/1.12.1/themes/base/jquery-ui.css" />
<link rel="stylesheet"
    href="https://cdn.datatables.net/1.10.19/css/jquery.dataTables.min.css"/>
<script>
    var myTable;
    $(document).ready(function() {
        $("#name").autocomplete({ source: "/autocomplete_name" });
        myTable = $("#personTable").DataTable({
        columns: [{data:"id"}, {data:"name"}, {data:"dept_name"}]
        });
    });
    function loadTableAsync() {
        var params = {persontype:$("#persontype").val(), name:$("#name").val()};
        var url = "/person_query_ajax?" + jQuery.param(params);
        myTable.ajax.url(url).load();
    }
</script>
</head> <body>
Search for:
<select id="persontype">
    <option value="student" selected>Student </option>
    <option value="instructor"> Instructor </option>
</select> <br>
Name: <input type=text size=20 id="name">
<button onclick="loadTableAsync()"> Show details </button>
<table id="personTable" border="1">
    <thead>
        <tr> <th>ID</th> <th>Name</th> <th>Dept. Name</th> </tr>
    </thead>
</table>
</body> </html>
```

Figure 9.12 HTML page using JavaScript and Ajax.

a state from a list of states in that country. Until the country is selected, the drop-down list of states is empty. The Ajax framework allows the list of states to be downloaded

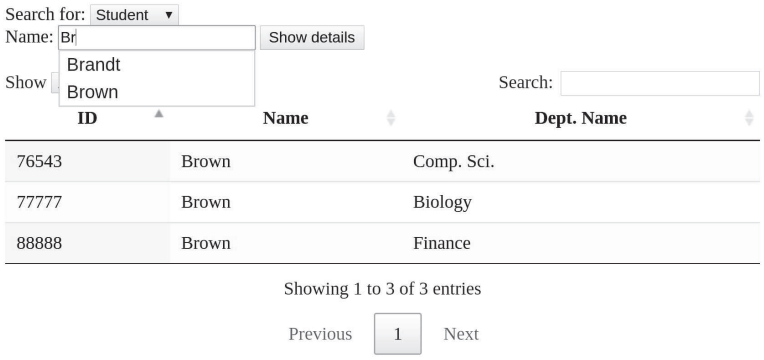


Figure 9.13 Screenshot of display generated by Figure 9.12.

from the web site in the background when the country is selected, and as soon as the list has been fetched, it is added to the drop-down list, which allows you to select the state.

9.5.2 Web Services

A **web service** is an application component that can be invoked over the web and functions, in effect, like an application programming interface. A web service request is sent using the HTTP protocol, it is executed at an application server, and the results are sent back to the calling program.

Two approaches are widely used to implement web services. In the simpler approach, called **Representation State Transfer** (or **REST**), web service function calls are executed by a standard HTTP request to a URL at an application server, with parameters sent as standard HTTP request parameters. The application server executes the request (which may involve updating the database at the server), generates and encodes the result, and returns the result as the result of the HTTP request. The most widely used encoding for the results today is the JSON representation, although XML, which we saw earlier in Section 8.1.3, is also used. The requestor parses the returned page to access the returned data.

In many applications of such RESTful web services (i.e., web services using REST), the requestor is JavaScript code running in a web browser; the code updates the browser screen using the result of the function call. For example, when you scroll the display on a map interface on the web, the part of the map that needs to be newly displayed may be fetched by JavaScript code using a RESTful interface and then displayed on the screen.

While some web services are not publicly documented and are used only internally by specific applications, other web services have their interfaces documented and can be used by any application. Such services may allow use without any restriction,

may require users to be logged in before accessing the service, or may require users or application developers to pay the web service provider for the privilege of using the service.

Today, a very large variety of RESTful web services are available, and most front-end applications use one or more such services to perform backend activities. For example, your web-based email system, your social media web page, or your web-based map service would almost surely be built with JavaScript code for rendering and would use backend web services to fetch data as well as to perform updates. Similarly, any mobile app that stores data at the back end almost surely uses web services to fetch data and to perform updates.

Web services are also increasingly used at the backend, to make use of functionalities provided by other backend systems. For example, web-based storage systems provide a web service API for storing and retrieving data; such services are provided by a number of providers, such as Amazon S3, Google Cloud Storage, and Microsoft Azure. They are very popular with application developers since they allow storage of very large amounts of data, and they support a very large number of operations per second, allowing scalability far beyond what a centralized database can support.

There are many more such web-service APIs. For example, text-to-speech, speech recognition, and vision web-service APIs allow developers to construct applications incorporating speech and image recognition with very little development effort.

A more complex and less frequently used approach, sometimes referred to as "Big Web Services," uses XML encoding of parameters as well as results, has a formal definition of the web API using a special language, and uses a protocol layer built on top of the HTTP protocol.

9.5.3 Disconnected Operation

Many applications wish to support some operations even when a client is disconnected from the application server. For example, a student may wish to complete an application form even if her laptop is disconnected from the network but have it saved back when the laptop is reconnected. As another example, if an email client is built as a web application, a user may wish to compose an email even if her laptop is disconnected from the network and have it sent when it is reconnected. Building such applications requires local storage in the client machine.

The HTML5 standard supports local storage, which can be accessed using JavaScript. The code:

```
if (typeof(Storage) !== "undefined") { // browser supports local storage
...
}
```

checks if the browser supports local storage. If it does, the following functions can be used to store, load, or delete values for a given key.

```
localStorage.setItem(key, value)
localStorage.getItem(key)
localStorage.deleteItem(key)
```

To avoid excessive data storage, the browser may limit a web site to storing at most some amount of data; the default maximum is typically 5 megabytes.

The above interface only allows storage/retrieval of key/value pairs. Retrieval requires that a key be provided; otherwise the entire set of key/value pairs will need to be scanned to find a required value. Applications may need to store tuples indexed on multiple attributes, allowing efficient access based on values of any of the attributes. HTML5 supports IndexedDB, which allows storage of JSON objects with indices on multiple attributes. IndexedDB also supports schema versions and allows the developer to provide code to migrate data from one schema version to the next version.

9.5.4 Mobile Application Platforms

Mobile applications (or mobile apps, for short) are widely used today, and they form the primary user interface for a large class of users. The two most widely used mobile platforms today are Android and iOS. Each of these platforms provides a way of building applications with a graphical user interface, tailored to small touch-screen devices. The graphical user interface provides a variety of standard GUI features such as menus, lists, buttons, check boxes, progress bars, and so on, and the ability to display text, images, and video.

Mobile apps can be downloaded and stored and used later. Thus, the user can download apps when connected to a high-speed network and then use the app with a lower-speed network. In contrast, web apps may get downloaded when they are used, resulting in a lot of data transfer when a user may be connected to a lower-speed network or a network where data transfer is expensive. Further, mobile apps can be better tuned to small-sized devices than web apps, with user interfaces that work well on small devices. Mobile apps can also be compiled to machine code, resulting in lower power demands than web apps. More importantly, unlike (earlier generation) web apps, mobile apps can store data locally, allowing offline usage. Further, mobile apps have a well-developed authorization model, allowing them to use information and device features such as location, cameras, contacts, and so on with user authorization.

However, one of the drawbacks of using mobile-app interfaces is that code written for the Android platform can only run on that platform and not on iOS, and vice versa. As a result, developers are forced to code every application twice, once for Android and once for iOS, unless they decide to ignore one of the platforms completely, which is not very desirable.

The ability to create applications where the same high-level code can run on either Android or iOS is clearly very important. The *React Native framework* based on JavaScript, developed by Facebook, and the *Flutter framework* based on the *Dart* language developed by Google, are designed to allow cross-platform development. (Dart

is a language optimized for developing user interfaces, providing features such as asynchronous function invocation and functions on streams.) Both frameworks allow much of the application code to be common for both Android and iOS, but some functionality can be made specific to the underlying platform in case it is not supported in the platform-independent part of the framework.

With the wide availability of high-speed mobile networks, some of the motivation for using mobile apps instead of web apps, such as the ability to download ahead of time, is not as important anymore. A new generation of web apps, called **Progressive Web Apps** (**PWA**) that combine the benefits of mobile apps with web apps is seeing increasing usage. Such apps are built using JavaScript and HTML5 and are tailored for mobile devices.

A key enabling feature for PWAs is the HTML5 support for local data storage, which allows apps to be used even when the device is offline. Another enabling feature is the support for compilation of JavaScript code; compilation is restricted to code that follows a restricted syntax, since compilation of arbitrary JavaScript code is not practical. Such compilation is typically done just-in-time, that is, it is done when the code needs to be executed, or if it has already been executed multiple times. Thus, by writing CPU-heavy parts of a web application using only JavaScript features that allow compilation, it is possible to ensure CPU and energy-efficient execution of the code on a mobile device.

PWAs also make use of HTML5 service workers, which allow a script to run in the background in the browser, separate from a web page. Such service workers can be used to perform background synchronization operations between the local store and a web service, or to receive or push notifications from a backend service. HTML5 also allows apps to get device location (after user authorization), allowing PWAs to use location information.

Thus, PWAs are likely to see increasing use, replacing many (but certainly not all) of the use cases for mobile apps.

9.6 Application Architectures

To handle their complexity, large applications are often broken into several layers:

- The *presentation* or *user-interface* layer, which deals with user interaction. A single application may have several different versions of this layer, corresponding to distinct kinds of interfaces such as web browsers and user interfaces of mobile phones, which have much smaller screens.

  In many implementations, the presentation/user-interface layer is itself conceptually broken up into layers, based on the **model-view-controller** (MVC) architecture. The **model** corresponds to the business-logic layer, described below. The **view** defines the presentation of data; a single underlying model can have different views depending on the specific software/device used to access the application. The **controller** receives events (user actions), executes actions on the model, and returns

a view to the user. The MVC architecture is used in a number of web application frameworks.

- The **business-logic** layer, which provides a high-level view of data and actions on data. We discuss the business-logic layer in more detail in Section 9.6.1.
- The **data-access** layer, which provides the interface between the business-logic layer and the underlying database. Many applications use an object-oriented language to code the business-logic layer and use an object-oriented model of data, while the underlying database is a relational database. In such cases, the data-access layer also provides the mapping from the object-oriented data model used by the business logic to the relational model supported by the database. We discuss such mappings in more detail in Section 9.6.2.

Figure 9.14 shows these layers, along with a sequence of steps taken to process a request from the web browser. The labels on the arrows in the figure indicate the order of the steps. When the request is received by the application server, the controller sends a request to the model. The model processes the request, using business logic, which may involve updating objects that are part of the model, followed by creating a result object. The model in turn uses the data-access layer to update or retrieve information from a database. The result object created by the model is sent to the view module, which creates an HTML view of the result to be displayed on the web browser. The view may be tailored based on the characteristics of the device used to view the result —for example, whether it is a computer monitor with a large screen or a small screen on a phone. Increasingly, the view layer is implemented by code running at the client, instead of at the server.

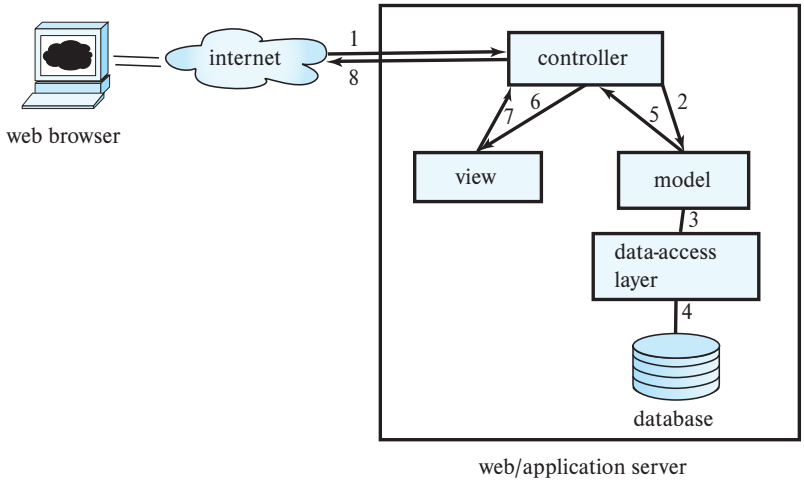


Figure 9.14 Web application architecture.

9.6.1 The Business-Logic Layer

The business-logic layer of an application for managing a university may provide abstractions of entities such as students, instructors, courses, sections, etc., and actions such as admitting a student to the university, enrolling a student in a course, and so on. The code implementing these actions ensures that **business rules** are satisfied; for example, the code would ensure that a student can enroll for a course only if she has already completed course prerequisites and has paid her tuition fees.

In addition, the business logic includes **workflows**, which describe how a particular task that involves multiple participants is handled. For example, if a candidate applies to the university, there is a workflow that defines who should see and approve the application first, and if approved in the first step, who should see the application next, and so on until either an offer is made to the student, or a rejection note is sent out. Workflow management also needs to deal with error situations; for example, if a deadline for approval/rejection is not met, a supervisor may need to be informed so she can intervene and ensure the application is processed.

9.6.2 The Data-Access Layer and Object-Relational Mapping

In the simplest scenario, where the business-logic layer uses the same data model as the database, the data-access layer simply hides the details of interfacing with the database. However, when the business-logic layer is written using an object-oriented programming language, it is natural to model data as objects, with methods invoked on objects.

In early implementations, programmers had to write code for creating objects by fetching data from the database and for storing updated objects back in the database. However, such manual conversions between data models is cumbersome and error prone. One approach to handling this problem was to develop a database system that natively stores objects, and relationships between objects, and allows objects in the database to be accessed in exactly the same way as in-memory objects. Such databases, called **object-oriented databases**, were discussed in Section 8.2. However, object-oriented databases did not achieve commercial success for a variety of technical and commercial reasons.

An alternative approach is to use traditional relational databases to store data, but to automate the mapping of data in relation to in-memory objects, which are created on demand (since memory is usually not sufficient to store all data in the database), as well as the reverse mapping to store updated objects back as relations in the database.

Several systems have been developed to implement such **object-relational mappings**. We describe the Hibernate and Django ORMs next.

9.6.2.1 Hibernate ORM

The **Hibernate** system is widely used for mapping from Java objects to relations. Hibernate provides an implementation of the Java Persistence API (JPA). In Hibernate, the mapping from each Java class to one or more relations is specified in a mapping file.

The mapping file can specify, for example, that a Java class called Student is mapped to the relation *student*, with the Java attribute ID mapped to the attribute *student*.ID, and so on. Information about the database, such as the host on which it is running and user name and password for connecting to the database, are specified in a *properties* file. The program has to open a *session*, which sets up the connection to the database. Once the session is set up, a Student object stud created in Java can be stored in the database by invoking session.save(stud). The Hibernate code generates the SQL commands required to store corresponding data in the *student* relation.

While entities in an E-R model naturally correspond to objects in an object-oriented language such as Java, relationships often do not. Hibernate supports the ability to map such relationships as sets associated with objects. For example, the *takes* relationship between *student* and *section* can be modeled by associating a set of *section*s with each *student*, and a set of *student*s with each *section*. Once the appropriate mapping is specified, Hibernate populates these sets automatically from the database relation *takes*, and updates to the sets are reflected back to the database relation on commit.

As an example of the use of Hibernate, we create a Java class corresponding to the *student* relation as follows:

```
@Entity public class Student {
        @Id String ID;
        String name;
        String department;
        int tot_cred;
}
```

To be precise, the class attributes should be declared as private, and getter/setter methods should be provided to access the attributes, but we omit these details.

The mapping of the class attributes of Student to attributes of the relation *student* can be specified in a mapping file, in an XML format, or more conveniently, by means of annotations of the Java code. In the example above, the annotation @Entity denotes that the class is mapped to a database relation, whose name by default is the class name, and whose attributes are by default the same as the class attributes. The default relation name and attribute names can be overridden using @Table and @Column annotations. The @Id annotation in the example specifies that ID is the primary key attribute.

The following code snippet then creates a Student object and saves it to the database.

```
Session session = getSessionFactory().openSession();
Transaction txn = session.beginTransaction();
Student stud = new Student("12328", "John Smith", "Comp. Sci.", 0);
session.save(stud);
txn.commit();
session.close();
```

Hibernate automatically generates the required SQL **insert** statement to create a *student* tuple in the database.

Objects can be retrieved either by primary key or by a query, as illustrated in the following code snippet:

```
Session session = getSessionFactory().openSession();
Transaction txn = session.beginTransaction();
// Retrieve student object by identifier
Student stud1 = session.get(Student.class, "12328");
        .. print out the Student information ..
List students =
        session.createQuery("from Student as s order by s.ID asc").list();
for ( Iterator iter = students.iterator(); iter.hasNext(); ) {
        Student stud = (Student) iter.next();
        .. print out the Student information ..
}
txn.commit();
session.close();
```

A single object can be retrieved using the session.get() method by providing its class and its primary key. The retrieved object can be updated in memory; when the transaction on the ongoing Hibernate session is committed, Hibernate automatically saves the updated objects by making corresponding updates on relations in the database.

The preceding code snippet also shows a query in Hibernate's HQL query language, which is based on SQL but designed to allow objects to be used directly in the query. The HQL query is automatically translated to SQL by Hibernate and executed, and the results are converted into a list of Student objects. The for loop iterates over the objects in this list.

These features help to provide the programmer with a high-level model of data without bothering about the details of the relational storage. However, Hibernate, like other object-relational mapping systems, also allows queries to be written using SQL on the relations stored in the database; such direct database access, bypassing the object model, can be quite useful for writing complex queries.

9.6.2.2 The Django ORM

Several ORMs have been developed for the Python language. The ORM component of the Django framework is one of the most popular such ORMs, while SQLAlchemy is another popular Python ORM.

Figure 9.15 shows a model definition for Student and Instructor in Django. Observe that all of the fields of *student* and *instructor* have been defined as fields in the class Student and Instructor, with appropriate type definitions.

In addition, the relation *advisor* has been modeled here as a many-to-many relationship between Student and Instructor. The relationship is accessed by an attribute

```
from django.db import models

class student(models.Model):
    id = models.CharField(primary_key=True, max_length=5)
    name = models.CharField(max_length=20)
    dept_name = models.CharField(max_length=20)
    tot_cred = models.DecimalField(max_digits=3, decimal_places=0)

class instructor(models.Model):
    id = models.CharField(primary_key=True, max_length=5)
    name = models.CharField(max_length=20)
    dept_name = models.CharField(max_length=20)
    salary = models.DecimalField(max_digits=8, decimal_places=2)
    advisees = models.ManyToManyField(student, related_name="advisors")
```

Figure 9.15 Model definition in Django.

called advisees in Instructor, which stores a set of references to Student objects. The reverse relationship from Student to Instructor is created automatically, and the model specifies that the reverse relationship attribute in the Student class is named advisors; this attribute stores a set of references to Instructor objects.

The Django view person_query_model shown in Figure 9.16 illustrates how to access database objects directly from the Python language, without using SQL. The expression Student.objects.filter() returns all student objects that satisfy the specified filter condition; in this case, students with the given name. The student names are printed out along with the names of their advisors. The expression Student.advisors.all() returns a list of advisors (advisor objects) of a given student, whose names are then retrieved and returned by the get_names() function. The case for instructors is similar, with instructor names being printed out along with the names of their advisees.

Django provides a tool called *migrate*, which creates database relations from a given model. Models can be given version numbers. When migrate is invoked on a model with a new version number, while an earlier version number is already in the database, the migrate tool also generates SQL code for migrating the existing data from the old database schema to the new database schema. It is also possible to create Django models from existing database schemas.

9.7 Application Performance

Web sites may be accessed by millions of people from across the globe, at rates of thousands of requests per second, or even more, for the most popular sites. Ensuring

```
from models import Student, Instructor
def get_names(persons):
    res = ""
    for p in persons:
        res += p.name + ", "
    return res.rstrip(", ")

def person_query_model(request):
    persontype = request.GET.get('persontype')
    personname = request.GET.get('personname')
    html = ""

    if persontype == 'student':
        students = Student.objects.filter(name=personname)
        for student in students:
            advisors = students.advisors.all()
            html = html + "Advisee: " + student.name + "<br>Advisors: "
                + get_names(advisors) + "<br> \n"
    else:
        instructors = Instructor.objects.filter(name=personname)
        for instructor in instructors:
            advisees = instructor.advisees.all()
            html = html+"Advisor: " + instructor.name + "<br>Advisees: "
                + get_names(advisees) + "<br> \n"
    return HttpResponse(html)
```

Figure 9.16 View definition in Django using models.

that requests are served with low response times is a major challenge for web-site developers. To do so, application developers try to speed up the processing of individual requests by using techniques such as caching, and they exploit parallel processing by using multiple application servers. We describe these techniques briefly next. Tuning of database applications is another way to improve performance and is described in Section 25.1.

9.7.1 Reducing Overhead by Caching

Suppose that the application code for servicing each user request connects to a database through JDBC. Creating a new JDBC connection may take several milliseconds, so opening a new connection for each user request is not a good idea if very high transaction rates are to be supported.

The **connection pooling** method is used to reduce this overhead; it works as follows: The connection pool manager (typically a part of the application server) creates a pool (that is, a set) of open ODBC/JDBC connections. Instead of opening a new connection to the database, the code servicing a user request (typically a servlet) asks for (requests) a connection from the connection pool and returns the connection to the pool when the code (servlet) completes its processing. If the pool has no unused connections when a connection is requested, a new connection is opened to the database (taking care not to exceed the maximum number of connections that the database system can support concurrently). If there are many open connections that have not been used for a while, the connection pool manager may close some of the open database connections. Many application servers and newer ODBC/JDBC drivers provide a built-in connection pool manager.

Details of how to create a connection pool vary by application server or JDBC driver, but most implementations require the creation of a `DataSource` object using the JDBC connection details such as the machine, port, database, user-id and password, as well as other parameters related to connection pooling. The `getConnection()` method invoked on the `DataSource` object gets a connection from the connection pool. Closing the connection returns the connection to the pool.

Certain requests may result in exactly the same query being resubmitted to the database. The cost of communication with the database can be greatly reduced by caching the results of earlier queries and reusing them, so long as the query result has not changed at the database. Some web servers support such query-result caching; caching can otherwise be done explicitly in application code.

Costs can be further reduced by caching the final web page that is sent in response to a request. If a new request comes with exactly the same parameters as a previous request, the request does not perform any updates, and the resultant web page is in the cache, that page can be reused to avoid the cost of recomputing the page. Caching can be done at the level of fragments of web pages, which are then assembled to create complete web pages.

Cached query results and cached web pages are forms of materialized views. If the underlying database data change, the cached results must be discarded, or recomputed, or even incrementally updated, as in materialized-view maintenance (described in Section 16.5). Some database systems (such as Microsoft SQL Server) provide a way for the application server to register a query with the database and get a **notification** from the database when the result of the query changes. Such a notification mechanism can be used to ensure that query results cached at the application server are up-to-date.

There are several widely used main-memory caching systems; among the more popular ones are **memcached** and **Redis**. Both systems allow applications to store data with an associated key and retrieve data for a specified key. Thus, they act as hash-map data structures that allow data to be stored in the main memory but also provide cache eviction of infrequently used data.

For example, with memcached, data can be stored using memcached.add(key, data) and fetched using memcached.fetch(key). Instead of issuing a database query to fetch user data with a specified key, say key1, from a relation r, an application would first check if the required data are already cached by issuing a fetch("r:"+key1) (here, the key is appended to the relation name, to distinguish data from different relations that may be stored in the same memcached instance). If the fetch returns null, the database query is issued, a copy of the data fetched from the database is stored in memcached, and the data are then returned to the user. If the fetch does find the requested data, it can be used without accessing the database, leading to much faster access.

A client can connect to multiple memcached instances, which may run on different machines and store/retrieve data from any of them. How to decide what data are stored on which instance is left to the client code. By partitioning the data storage across multiple machines, an application can benefit from the aggregate main memory available across all the machines.

Memcached does not support automatic invalidation of cached data, but the application can track database changes and issue updates (using memcached_set(key, newvalue)) or deletes (using memcached_delete(key)) for the key values affected by update or deletion in the database. Redis offers very similar functionality. Both memcached and Redis provide APIs in multiple languages.

9.7.2 Parallel Processing

A commonly used approach to handling such very heavy loads is to use a large number of application servers running in parallel, each handling a fraction of the requests. A web server or a network router can be used to route each client request to one of the application servers. All requests from a particular client session must go to the same application server, since the server maintains state for a client session. This property can be ensured, for example, by routing all requests from a particular IP address to the same application server. The underlying database is, however, shared by all the application servers, so users see a consistent view of the database.

While the above architecture ensures that application servers do not become bottlenecks, it cannot prevent the database from becoming a bottleneck, since there is only one database server. To avoid overloading the database, application designers often use caching techniques to reduce the number of requests to the database. In addition, parallel database systems, described in Chapter 21 through Chapter 23, are used when the database needs to handle very large amounts of data, or a very large query load. Parallel data storage systems that are accessible via web service APIs are also popular in applications that need to scale to a very large number of users.

9.8 Application Security

Application security has to deal with several security threats and issues beyond those handled by SQL authorization.

The first point where security has to be enforced is in the application. To do so, applications must authenticate users and ensure that users are only allowed to carry out authorized tasks.

There are many ways in which an application's security can be compromised, even if the database system is itself secure, due to badly written application code. In this section, we first describe several security loopholes that can permit hackers to carry out actions that bypass the authentication and authorization checks carried out by the application, and we explain how to prevent such loopholes. Later in the section, we describe techniques for secure authentication, and for fine-grained authorization. We then describe audit trails that can help in recovering from unauthorized access and from erroneous updates. We conclude the section by describing issues in data privacy.

9.8.1 SQL Injection

In **SQL injection** attacks, the attacker manages to get an application to execute an SQL query created by the attacker. In Section 5.1.1.5, we saw an example of an SQL injection vulnerability if user inputs are concatenated directly with an SQL query and submitted to the database. As another example of SQL injection vulnerability, consider the form source text shown in Figure 9.3. Suppose the corresponding servlet shown in Figure 9.7 creates an SQL query string using the following Java expression:

```
String query = "select * from student where name like '%"
                + name + "%' "
```

where name is a variable containing the string input by the user, and then executes the query on the database. A malicious attacker using the web form can then type a string such as " '; <some SQL statement>; -- ", where <some SQL statement> denotes any SQL statement that the attacker desires, in place of a valid student name. The servlet would then execute the following string.

```
select * from student where name like '%'; <some SQL statement>; -- %'
```

The quote inserted by the attacker closes the string, the following semicolon terminates the query, and the following text inserted by the attacker gets interpreted as a second SQL query, while the closing quote has been commented out. Thus, the malicious user has managed to insert an arbitrary SQL statement that is executed by the application. The statement can cause significant damage, since it can perform any action on the database, bypassing all security measures implemented in the application code.

As discussed in Section 5.1.1.5, to avoid such attacks, it is best to use prepared statements to execute SQL queries. When setting a parameter of a prepared query, JDBC automatically adds escape characters so that the user-supplied quote would no longer be able to terminate the string. Equivalently, a function that adds such escape

characters could be applied on input strings before they are concatenated with the SQL query, instead of using prepared statements.

Another source of SQL-injection risk comes from applications that create queries dynamically, based on selection conditions and ordering attributes specified in a form. For example, an application may allow a user to specify what attribute should be used for sorting the results of a query. An appropriate SQL query is constructed, based on the attribute specified. Suppose the application takes the attribute name from a form, in the variable orderAttribute, and creates a query string such as the following:

```
String query = "select * from takes order by " + orderAttribute;
```

A malicious user can send an arbitrary string in place of a meaningful orderAttribute value, even if the HTML form used to get the input tried to restrict the allowed values by providing a menu. To avoid this kind of SQL injection, the application should ensure that the orderAttribute variable value is one of the allowed values (in our example, attribute names) before appending it.

9.8.2 Cross-Site Scripting and Request Forgery

A web site that allows users to enter text, such as a comment or a name, and then stores it and later displays it to other users, is potentially vulnerable to a kind of attack called a **cross-site scripting** (**XSS**) attack. In such an attack, a malicious user enters code written in a client-side scripting language such as JavaScript or Flash instead of entering a valid name or comment. When a different user views the entered text, the browser executes the script, which can carry out actions such as sending private cookie information back to the malicious user or even executing an action on a different web server that the user may be logged into.

For example, suppose the user happens to be logged into her bank account at the time the script executes. The script could send cookie information related to the bank account login back to the malicious user, who could use the information to connect to the bank's web server, fooling it into believing that the connection is from the original user. Or the script could access appropriate pages on the bank's web site, with appropriately set parameters, to execute a money transfer. In fact, this particular problem can occur even without scripting by simply using a line of code such as

```
<img src=
    "https://mybank.com/transfermoney?amount=1000&toaccount=14523">
```

assuming that the URL mybank.com/transfermoney accepts the specified parameters and carries out a money transfer. This latter kind of vulnerability is also called **cross-site request forgery** or **XSRF** (sometimes also called **CSRF**).

XSS can be done in other ways, such as luring a user into visiting a web site that has malicious scripts embedded in its pages. There are other more complex kinds of XSS

and XSRF attacks, which we shall not get into here. To protect against such attacks, two things need to be done:

- **Prevent your web site from being used to launch XSS or XSRF attacks.**
  The simplest technique is to disallow any HTML tags whatsoever in text input by users. There are functions that detect or strip all such tags. These functions can be used to prevent HTML tags, and as a result, any scripts, from being displayed to other users. In some cases HTML formatting is useful, and in that case functions that parse the text and allow limited HTML constructs but disallow other dangerous constructs can be used instead; these must be designed carefully, since something as innocuous as an image include could potentially be dangerous in case there is a bug in the image display software that can be exploited.
- **Protect your web site from XSS or XSRF attacks launched from other sites.**
  If the user has logged into your web site and visits a different web site vulnerable to XSS, the malicious code executing on the user's browser could execute actions on your web site or pass session information related to your web site back to the malicious user, who could try to exploit it. This cannot be prevented altogether, but you can take a few steps to minimize the risk.
  - The HTTP protocol allows a server to check the **referer** of a page access, that is, the URL of the page that had the link that the user clicked on to initiate the page access. By checking that the referer is valid, for example, that the referer URL is a page on the same web site, XSS attacks that originated on a different web page accessed by the user can be prevented.
  - Instead of using only the cookie to identify a session, the session could also be restricted to the IP address from which it was originally authenticated. As a result, even if a malicious user gets a cookie, he may not be able to log in from a different computer.
  - Never use a `GET` method to perform any updates. This prevents attacks using `<img src ..>` such as the one we saw earlier. In fact, the HTTP standard specifies that `GET` methods should not perform any updates.
  - If you use a web application framework like Django, make sure to use the XSRF/CSRF protection mechanisms provided by the framework.

9.8.3 Password Leakage

Another problem that application developers must deal with is storing passwords in clear text in the application code. For example, programs such as JSP scripts often contain passwords in clear text. If such scripts are stored in a directory accessible by a web server, an external user may be able to access the source code of the script and get access to the password for the database account used by the application. To avoid such problems, many application servers provide mechanisms to store passwords in

encrypted form, which the server decrypts before passing it on to the database. Such a feature removes the need for storing passwords as clear text in application programs. However, if the decryption key is also vulnerable to being exposed, this approach is not fully effective.

As another measure against compromised database passwords, many database systems allow access to the database to be restricted to a given set of internet addresses, typically, the machines running the application servers. Attempts to connect to the database from other internet addresses are rejected. Thus, unless the malicious user is able to log into the application server, she cannot do any damage even if she gains access to the database password.

9.8.4 Application-Level Authentication

Authentication refers to the task of verifying the identity of a person/software connecting to an application. The simplest form of authentication consists of a secret password that must be presented when a user connects to the application. Unfortunately, passwords are easily compromised, for example, by guessing, or by sniffing of packets on the network if the passwords are not sent encrypted. More robust schemes are needed for critical applications, such as online bank accounts. Encryption is the basis for more robust authentication schemes. Authentication through encryption is addressed in Section 9.9.3.

Many applications use **two-factor authentication**, where two independent *factors* (i.e., pieces of information or processes) are used to identify a user. The two factors should not share a common vulnerability; for example, if a system merely required two passwords, both could be vulnerable to leakage in the same manner (by network sniffing, or by a virus on the computer used by the user, for example). While biometrics such as fingerprints or iris scanners can be used in situations where a user is physically present at the point of authentication, they are not very meaningful across a network.

Passwords are used as the first factor in most such two-factor authentication schemes. Smart cards or other encryption devices connected through the USB interface, which can be used for authentication based on encryption techniques (see Section 9.9.3), are widely used as second factors.

One-time password devices, which generate a new pseudo-random number (say) every minute are also widely used as a second factor. Each user is given one of these devices and must enter the number displayed by the device at the time of authentication, along with the password, to authenticate himself. Each device generates a different sequence of pseudo-random numbers. The application server can generate the same sequence of pseudo-random numbers as the device given to the user, stopping at the number that would be displayed at the time of authentication, and verify that the numbers match. This scheme requires that the clock in the device and at the server are synchronized reasonably closely.

Yet another second-factor approach is to send an SMS with a (randomly generated) one-time password to the user's phone (whose number is registered earlier) whenever

the user wishes to log in to the application. The user must possess a phone with that number to receive the SMS and then enter the one-time password, along with her regular password, to be authenticated.

It is worth noting that even with two-factor authentication, users may still be vulnerable to **man-in-the-middle attacks**. In such attacks, a user attempting to connect to the application is diverted to a fake web site, which accepts the password (including second factor passwords) from the user and uses it immediately to authenticate to the original application. The HTTPS protocol, described in Section 9.9.3.2, is used to authenticate the web site to the user (so the user does not connect to a fake site believing it to be the intended site). The HTTPS protocol also encrypts data and prevents man-in-the-middle attacks.

When users access multiple web sites, it is often annoying for a user to have to authenticate herself to each site separately, often with different passwords on each site. There are systems that allow the user to authenticate herself to one central authentication service, and other web sites and applications can authenticate the user through the central authentication service; the same password can then be used to access multiple sites. The LDAP protocol is widely used to implement such a central point of authentication for applications within a single organization; organizations implement an LDAP server containing user names and password information, and applications use the LDAP server to authenticate users.

In addition to authenticating users, a central authentication service can provide other services, for example, providing information about the user such as name, email, and address information, to the application. This obviates the need to enter this information separately in each application. LDAP can be used for this task, as described in Section 25.5.2. Other directory systems such Microsoft's Active Directories also provide mechanisms for authenticating users as well as for providing user information.

A **single sign-on** system further allows the user to be authenticated once, and multiple applications can then verify the user's identity through an authentication service without requiring reauthentication. In other words, once a user is logged in at one site, he does not have to enter his user name and password at other sites that use the same single sign-on service. Such single sign-on mechanisms have long been used in network authentication protocols such as Kerberos, and implementations are now available for web applications.

The **Security Assertion Markup Language** (**SAML**) is a protocol for exchanging authentication and authorization information between different security domains, to provide cross-organization single sign-on. For example, suppose an application needs to provide access to all students from a particular university, say Yale. The university can set up a web-based service that carries out authentication. Suppose a user connects to the application with a username such as "joe@yale.edu". The application, instead of directly authenticating a user, diverts the user to Yale University's authentication service, which authenticates the user and then tells the application who the user is and

may provide some additional information such as the category of the user (student or instructor) or other relevant information. The user's password and other authentication factors are never revealed to the application, and the user need not register explicitly with the application. However, the application must trust the university's authentication service when authenticating a user.

The **OpenID** protocol is an alternative for single sign-on across organizations, which works in a manner similar to SAML. The **OAuth** protocol is another protocol that allows users to authorize access to certain resources, via sharing of an authorization token.

9.8.5 Application-Level Authorization

Although the SQL standard supports a fairly flexible system of authorization based on roles (described in Section 4.7), the SQL authorization model plays a very limited role in managing user authorizations in a typical application. For instance, suppose you want all students to be able to see their own grades, but not the grades of anyone else. Such authorization cannot be specified in SQL for at least two reasons:

1. **Lack of end-user information**. With the growth in the web, database accesses come primarily from web application servers. The end users typically do not have individual user identifiers on the database itself, and indeed there may only be a single user identifier in the database corresponding to all users of an application server. Thus, authorization specification in SQL cannot be used in the above scenario.

   It is possible for an application server to authenticate end users and then pass the authentication information on to the database. In this section we will assume that the function *syscontext.user_id*() returns the identifier of the application user on whose behalf a query is being executed. [6]

2. **Lack of fine-grained authorization**. Authorization must be at the level of individual tuples if we are to authorize students to see only their own grades. Such authorization is not possible in the current SQL standard, which permits authorization only on an entire relation or view, or on specified attributes of relations or views.

   We could try to get around this limitation by creating for each student a view on the *takes* relation that shows only that student's grades. While this would work in principle, it would be extremely cumbersome since we would have to create one such view for every single student enrolled in the university, which is completely impractical.[7]

   An alternative is to create a view of the form

[6] In Oracle, a JDBC connection using Oracle's JDBC drivers can set the end user identifier using the method `OracleConnection.setClientIdentifier(userId)`, and an SQL query can use the function `sys_context('USERENV', 'CLIENT_IDENTIFIER')` to retrieve the user identifier.

[7] Database systems are designed to manage large relations but to manage schema information such as views in a way that assumes smaller data volumes so as to enhance overall performance.

```
create view studentTakes as
select *
from takes
where takes.ID= syscontext.user_id()
```

Users are then given authorization to this view, rather than to the underlying *takes* relation. However, queries executed on behalf of students must now be written on the view *studentTakes*, rather than on the original *takes* relation, whereas queries executed on behalf of instructors may need to use a different view. The task of developing applications becomes more complex as a result.

The task of authorization is often typically carried out entirely in the application, bypassing the authorization facilities of SQL. At the application level, users are authorized to access specific interfaces, and they may further be restricted to view or update certain data items only.

While carrying out authorization in the application gives a great deal of flexibility to application developers, there are problems, too.

- The code for checking authorization becomes intermixed with the rest of the application code.
- Implementing authorization through application code, rather than specifying it declaratively in SQL, makes it hard to ensure the absence of loopholes. Because of an oversight, one of the application programs may not check for authorization, allowing unauthorized users access to confidential data.

Verifying that all application programs make all required authorization checks involves reading through all the application-server code, a formidable task in a large system. In other words, applications have a very large "surface area," making the task of protecting the application significantly harder. And in fact, security loopholes have been found in a variety of real-life applications.

In contrast, if a database directly supported fine-grained authorization, authorization policies could be specified and enforced at the SQL level, which has a much smaller surface area. Even if some of the application interfaces inadvertently omit required authorization checks, the SQL-level authorization could prevent unauthorized actions from being executed.

Some database systems provide mechanisms for row-level authorization as we saw in Section 4.7.7. For example, the Oracle **Virtual Private Database** (**VPD**) allows a system administrator to associate a function with a relation; the function returns a predicate that must be added to any query that uses the relation (different functions can be defined for relations that are being updated). For example, using our syntax for retrieving application user identifiers, the function for the *takes* relation can return a predicate such as:

$$ID = sys_context.user_id()$$

This predicate is added to the **where** clause of every query that uses the *takes* relation. As a result (assuming that the application program sets the *user_id* value to the student's *ID*), each student can see only the tuples corresponding to courses that she took.

As we discussed in Section 4.7.7, a potential pitfall with adding a predicate as described above is that it may change the meaning of a query. For example, if a user wrote a query to find the average grade over all courses, she would end up getting the average of *her* grades, not all grades. Although the system would give the "right" answer for the rewritten query, that answer would not correspond to the query the user may have thought she was submitting.

PostgreSQL and Microsoft SQL Server offer row-level authorization support with similar functionality to Oracle VPD. More information on Oracle VPD and PostgreSQL and SQL Server row-level authorization may be found in their respective system manuals available online.

9.8.6 Audit Trails

An **audit trail** is a log of all changes (inserts, deletes, and updates) to the application data, along with information such as which user performed the change and when the change was performed. If application security is breached, or even if security was not breached, but some update was carried out erroneously, an audit trail can (a) help find out what happened, and who may have carried out the actions, and (b) aid in fixing the damage caused by the security breach or erroneous update.

For example, if a student's grade is found to be incorrect, the audit log can be examined to locate when and how the grade was updated, as well as to find which user carried out the updates. The university could then also use the audit trail to trace all the updates performed by this user in order to find other incorrect or fraudulent updates, and then correct them.

Audit trails can also be used to detect security breaches where a user's account is compromised and accessed by an intruder. For example, each time a user logs in, she may be informed about all updates in the audit trail that were done from that login in the recent past; if the user sees an update that she did not carry out, it is likely the account has been compromised.

It is possible to create a database-level audit trail by defining appropriate triggers on relation updates (using system-defined variables that identify the user name and time). However, many database systems provide built-in mechanisms to create audit trails that are much more convenient to use. Details of how to create audit trails vary across database systems, and you should refer to the database-system manuals for details.

Database-level audit trails are usually insufficient for applications, since they are usually unable to track who was the end user of the application. Further, updates are recorded at a low level, in terms of updates to tuples of a relation, rather than at a higher level, in terms of the business logic. Applications, therefore, usually create a

higher-level audit trail, recording, for example, what action was carried out, by whom, when, and from which IP address the request originated.

A related issue is that of protecting the audit trail itself from being modified or deleted by users who breach application security. One possible solution is to copy the audit trail to a different machine, to which the intruder would not have access, with each record in the trail copied as soon as it is generated. A more robust solution is to use blockchain techniques, which are described in Chapter 26; blockchain techniques store logs in multiple machines and use a hashing mechanism that makes it very difficult for an intruder to modify or delete data without being detected.

9.8.7 Privacy

In a world where an increasing amount of personal data are available online, people are increasingly worried about the privacy of their data. For example, most people would want their personal medical data to be kept private and not revealed publicly. However, the medical data must be made available to doctors and emergency medical technicians who treat the patient. Many countries have laws on privacy of such data that define when and to whom the data may be revealed. Violation of privacy law can result in criminal penalties in some countries. Applications that access such private data must be built carefully, keeping the privacy laws in mind.

On the other hand, aggregated private data can play an important role in many tasks such as detecting drug side effects, or in detecting the spread of epidemics. How to make such data available to researchers carrying out such tasks without compromising the privacy of individuals is an important real-world problem. As an example, suppose a hospital hides the name of the patient but provides a researcher with the date of birth and the postal code of the patient (both of which may be useful to the researcher). Just these two pieces of information can be used to uniquely identify the patient in many cases (using information from an external database), compromising his privacy. In this particular situation, one solution would be to give the year of birth but not the date of birth, along with the address, which may suffice for the researcher. This would not provide enough information to uniquely identify most individuals.[8]

As another example, web sites often collect personal data such as address, telephone, email, and credit-card information. Such information may be required to carry out a transaction such as purchasing an item from a store. However, the customer may not want the information to be made available to other organizations, or may want part of the information (such as credit-card numbers) to be erased after some period of time as a way to prevent it from falling into unauthorized hands in the event of a security breach. Many web sites allow customers to specify their privacy preferences, and those web sites must then ensure that these preferences are respected.

[8]For extremely old people, who are relatively rare, even the year of birth plus postal code may be enough to uniquely identify the individual, so a range of values, such as 90 years or older, may be provided instead of the actual age for people older than 90 years.

9.9 Encryption and Its Applications

Encryption refers to the process of transforming data into a form that is unreadable, unless the reverse process of decryption is applied. Encryption algorithms use an encryption key to perform encryption, and they require a decryption key (which could be the same as the encryption key, depending on the encryption algorithm used) to perform decryption.

The oldest uses of encryption were for transmitting messages, encrypted using a secret key known only to the sender and the intended receiver. Even if the message is intercepted by an enemy, the enemy, not knowing the key, will not be able to decrypt and understand the message. Encryption is widely used today for protecting data in transit in a variety of applications such as data transfer on the internet, and on cell-phone networks. Encryption is also used to carry out other tasks, such as authentication, as we will see in Section 9.9.3.

In the context of databases, encryption is used to store data in a secure way, so that even if the data are acquired by an unauthorized user (e.g., a laptop computer containing the data is stolen), the data will not be accessible without a decryption key.

Many databases today store sensitive customer information, such as credit-card numbers, names, fingerprints, signatures, and identification numbers such as, in the United States, social security numbers. A criminal who gets access to such data can use them for a variety of illegal activities, such as purchasing goods using a credit-card number, or even acquiring a credit card in someone else's name. Organizations such as credit-card companies use knowledge of personal information as a way of identifying who is requesting a service or goods. Leakage of such personal information allows a criminal to impersonate someone else and get access to service or goods; such impersonation is referred to as **identity theft**. Thus, applications that store such sensitive data must take great care to protect them from theft.

To reduce the chance of sensitive information being acquired by criminals, many countries and states today require by law that any database storing such sensitive information must store the information in an encrypted form. A business that does not protect its data thus could be held criminally liable in case of data theft. Thus, encryption is a critical component of any application that stores such sensitive information.

9.9.1 Encryption Techniques

There are a vast number of techniques for the encryption of data. Simple encryption techniques may not provide adequate security, since it may be easy for an unauthorized user to break the code. As an example of a weak encryption technique, consider the substitution of each character with the next character in the alphabet. Thus,

Perryridge

becomes

Qfsszsjehf

If an unauthorized user sees only "Qfsszsjehf," she probably has insufficient information to break the code. However, if the intruder sees a large number of encrypted branch names, she could use statistical data regarding the relative frequency of characters to guess what substitution is being made (for example, *E* is the most common letter in English text, followed by *T, A, O, N, I*, and so on).

A good encryption technique has the following properties:

- It is relatively simple for authorized users to encrypt and decrypt data.
- It depends not on the secrecy of the algorithm, but rather on a parameter of the algorithm called the *encryption key*, which is used to encrypt data. In a **symmetric-key** encryption technique, the encryption key is also used to decrypt data. In contrast, in **public-key** (also known as **asymmetric-key**) encryption techniques, there are two different keys, the public key and the private key, used to encrypt and decrypt the data.
- Its decryption key is extremely difficult for an intruder to determine, even if the intruder has access to encrypted data. In the case of asymmetric-key encryption, it is extremely difficult to infer the private key even if the public key is available.

The **Advanced Encryption Standard** (**AES**) is a symmetric-key encryption algorithm that was adopted as an encryption standard by the U.S. government in 2000 and is now widely used. The standard is based on the **Rijndael algorithm** (named for the inventors V. Rijmen and J. Daemen). The algorithm operates on a 128-bit block of data at a time, while the key can be 128, 192, or 256 bits in length. The algorithm runs a series of steps to jumble up the bits in a data block in a way that can be reversed during decryption, and it performs an XOR operation with a 128-bit "round key" that is derived from the encryption key. A new round key is generated from the encryption key for each block of data that is encrypted. During decryption, the round keys are generated again from the encryption key and the encryption process is reversed to recover the original data. An earlier standard called the *Data Encryption Standard* (DES), adopted in 1977, was very widely used earlier.

For any symmetric-key encryption scheme to work, authorized users must be provided with the encryption key via a secure mechanism. This requirement is a major weakness, since the scheme is no more secure than the security of the mechanism by which the encryption key is transmitted.

Public-key encryption is an alternative scheme that avoids some of the problems faced by symmetric-key encryption techniques. It is based on two keys: a *public key* and a *private key*. Each user U_i has a public key E_i and a private key D_i. All public keys are published: They can be seen by anyone. Each private key is known to only the one user to whom the key belongs. If user U_1 wants to store encrypted data, U_1 encrypts them using public key E_1. Decryption requires the private key D_1.

Because the encryption key for each user is public, it is possible to exchange information securely by this scheme. If user U_1 wants to share data with U_2, U_1 encrypts

the data using E_2, the public key of U_2. Since only user U_2 knows how to decrypt the data, information can be transferred securely.

For public-key encryption to work, there must be a scheme for encryption such that it is infeasible (that is, extremely hard) to deduce the private key, given the public key. Such a scheme does exist and is based on these conditions:

- There is an efficient algorithm for testing whether or not a number is prime.
- No efficient algorithm is known for finding the prime factors of a number.

For purposes of this scheme, data are treated as a collection of integers. We create a public key by computing the product of two large prime numbers: P_1 and P_2. The private key consists of the pair (P_1, P_2). The decryption algorithm cannot be used successfully if only the product P_1P_2 is known; it needs the individual values P_1 and P_2. Since all that is published is the product P_1P_2, an unauthorized user would need to be able to factor P_1P_2 to steal data. By choosing P_1 and P_2 to be sufficiently large (over 100 digits), we can make the cost of factoring P_1P_2 prohibitively high (on the order of years of computation time, on even the fastest computers).

The details of public-key encryption and the mathematical justification of this technique's properties are referenced in the bibliographical notes.

Although public-key encryption by this scheme is secure, it is also computationally very expensive. A hybrid scheme widely used for secure communication is as follows: a symmetric encryption key (based, for example, on AES) is randomly generated and exchanged in a secure manner using a public-key encryption scheme, and symmetric-key encryption using that key is used on the data transmitted subsequently.

Encryption of small values, such as identifiers or names, is made complicated by the possibility of **dictionary attacks**, particularly if the encryption key is publicly available. For example, if date-of-birth fields are encrypted, an attacker trying to decrypt a particular encrypted value e could try encrypting every possible date of birth until he finds one whose encrypted value matches e. Even if the encryption key is not publicly available, statistical information about data distributions can be used to figure out what an encrypted value represents in some cases, such as age or address. For example, if the age 18 is the most common age in a database, the encrypted age value that occurs most often can be inferred to represent 18.

Dictionary attacks can be deterred by adding extra random bits to the end of the value before encryption (and removing them after decryption). Such extra bits, referred to as an **initialization vector** in AES, or as *salt* bits in other contexts, provide good protection against dictionary attack.

9.9.2 Encryption Support in Databases

Many file systems and database systems today support encryption of data. Such encryption protects the data from someone who is able to access the data but is not able to access the decryption key. In the case of file-system encryption, the data to be encrypted are usually large files and directories containing information about files.

In the context of databases, encryption can be done at several different levels. At the lowest level, the disk blocks containing database data can be encrypted, using a key available to the database-system software. When a block is retrieved from disk, it is first decrypted and then used in the usual fashion. Such disk-block-level encryption protects against attackers who can access the disk contents but do not have access to the encryption key.

At the next higher level, specified (or all) attributes of a relation can be stored in encrypted form. In this case, each attribute of a relation could have a different encryption key. Many databases today support encryption at the level of specified attributes as well as at the level of an entire relation, or all relations in a database. Encryption of specified attributes minimizes the overhead of decryption by allowing applications to encrypt only attributes that contain sensitive values such as credit-card numbers. Encryption also then needs to use extra random bits to prevent dictionary attacks, as described earlier. However, databases typically do not allow primary and foreign key attributes to be encrypted, and they do not support indexing on encrypted attributes.

A decryption key is obviously required to get access to encrypted data. A single master encryption key may be used for all the encrypted data; with attribute level encryption, different encryption keys could be used for different attributes. In this case, the decryption keys for different attributes can be stored in a file or relation (often referred to as "wallet"), which is itself encrypted using a master key.

A connection to the database that needs to access encrypted attributes must then provide the master key; unless this is provided, the connection will not be able to access encrypted data. The master key would be stored in the application program (typically on a different computer), or memorized by the database user, and provided when the user connects to the database.

Encryption at the database level has the advantage of requiring relatively low time and space overhead and does not require modification of applications. For example, if data in a laptop computer database need to be protected from theft of the computer itself, such encryption can be used. Similarly, someone who gets access to backup tapes of a database would not be able to access the data contained in the backups without knowing the decryption key.

An alternative to performing encryption in the database is to perform it *before* the data are sent to the database. The application must then encrypt the data before sending it to the database and decrypt the data when they are retrieved. This approach to data encryption requires significant modifications to be done to the application, unlike encryption performed in a database system.

9.9.3 Encryption and Authentication

Password-based authentication is used widely by operating systems as well as database systems. However, the use of passwords has some drawbacks, especially over a network. If an eavesdropper is able to "sniff" the data being sent over the network, she may be able to find the password as it is being sent across the network. Once the eavesdropper

has a user name and password, she can connect to the database, pretending to be the legitimate user.

A more secure scheme involves a **challenge-response** system. The database system sends a challenge string to the user. The user encrypts the challenge string using a secret password as encryption key and then returns the result. The database system can verify the authenticity of the user by decrypting the string with the same secret password and checking the result with the original challenge string. This scheme ensures that no passwords travel across the network.

Public-key systems can be used for encryption in challenge – response systems. The database system encrypts a challenge string using the user's public key and sends it to the user. The user decrypts the string using her private key and returns the result to the database system. The database system then checks the response. This scheme has the added benefit of not storing the secret password in the database, where it could potentially be seen by system administrators.

Storing the private key of a user on a computer (even a personal computer) has the risk that if the computer is compromised, the key may be revealed to an attacker who can then masquerade as the user. **Smart cards** provide a solution to this problem. In a smart card, the key can be stored on an embedded chip; the operating system of the smart card guarantees that the key can never be read, but it allows data to be sent to the card for encryption or decryption, using the private key.[9]

9.9.3.1 Digital Signatures

Another interesting application of public-key encryption is in **digital signatures** to verify authenticity of data; digital signatures play the electronic role of physical signatures on documents. The private key is used to "sign," that is, encrypt, data, and the signed data can be made public. Anyone can verify the signature by decrypting the data using the public key, but no one could have generated the signed data without having the private key. (Note the reversal of the roles of the public and private keys in this scheme.) Thus, we can **authenticate** the data; that is, we can verify that the data were indeed created by the person who is supposed to have created them.

Furthermore, digital signatures also serve to ensure **nonrepudiation**. That is, in case the person who created the data later claims she did not create them (the electronic equivalent of claiming not to have signed the check), we can prove that that person must have created the data (unless her private key was leaked to others).

9.9.3.2 Digital Certificates

Authentication is, in general, a two-way process, where each of a pair of interacting entities authenticates itself to the other. Such pairwise authentication is needed even

[9] Smart cards provide other functionality too, such as the ability to store cash digitally and make payments, which is not relevant in our context.

when a client contacts a web site, to prevent a malicious site from masquerading as a legal web site. Such masquerading could be done, for example, if the network routers were compromised and data rerouted to the malicious site.

For a user to ensure that she is interacting with an authentic web site, she must have the site's public key. This raises the problem of how the user can get the public key—if it is stored on the web site, the malicious site could supply a different key, and the user would have no way of verifying if the supplied public key is itself authentic. Authentication can be handled by a system of **digital certificates**, whereby public keys are signed by a certification agency, whose public key is well known. For example, the public keys of the root certification authorities are stored in standard web browsers. A certificate issued by them can be verified by using the stored public keys.

A two-level system would place an excessive burden of creating certificates on the root certification authorities, so a multilevel system is used instead, with one or more root certification authorities and a tree of certification authorities below each root. Each authority (other than the root authorities) has a digital certificate issued by its parent.

A digital certificate issued by a certification authority A consists of a public key K_A and an encrypted text E that can be decoded by using the public key K_A. The encrypted text contains the name of the party to whom the certificate was issued and her public key K_c. In case the certification authority A is not a root certification authority, the encrypted text also contains the digital certificate issued to A by its parent certification authority; this certificate authenticates the key K_A itself. (That certificate may in turn contain a certificate from a further parent authority, and so on.)

To verify a certificate, the encrypted text E is decrypted by using the public key K_A to retrieve the name of the party (i.e., the name of the organization owning the web site); additionally, if A is not a root authority whose public key is known to the verifier, the public key K_A is verified recursively by using the digital certificate contained within E; recursion terminates when a certificate issued by the root authority is reached. Verifying the certificate establishes the chain through which a particular site was authenticated and provides the name and authenticated public key for the site.

Digital certificates are widely used to authenticate web sites to users, to prevent malicious sites from masquerading as other web sites. In the HTTPS protocol (the secure version of the HTTP protocol), the site provides its digital certificate to the browser, which then displays it to the user. If the user accepts the certificate, the browser then uses the provided public key to encrypt data. A malicious site will have access to the certificate, but not the private key, and will thus not be able to decrypt the data sent by the browser. Only the authentic site, which has the corresponding private key, can decrypt the data sent by the browser. We note that public-/private-key encryption and decryption costs are much higher than encryption/decryption costs using symmetric private keys. To reduce encryption costs, HTTPS actually creates a one-time symmetric key after authentication and uses it to encrypt data for the rest of the session.

Digital certificates can also be used for authenticating users. The user must submit a digital certificate containing her public key to a site, which verifies that the certificate has been signed by a trusted authority. The user's public key can then be used in a challenge-response system to ensure that the user possesses the corresponding private key, thereby authenticating the user.

9.10 Summary

- Application programs that use databases as back ends and interact with users have been around since the 1960s. Application architectures have evolved over this period. Today most applications use web browsers as their front end, and a database as their back end, with an application server in between.
- HTML provides the ability to define interfaces that combine hyperlinks with forms facilities. Web browsers communicate with web servers by the HTTP protocol. Web servers can pass on requests to application programs and return the results to the browser.
- Web servers execute application programs to implement desired functionality. Servlets are a widely used mechanism to write application programs that run as part of the web server process, in order to reduce overhead. There are also many server-side scripting languages that are interpreted by the web server and provide application-program functionality as part of the web server.
- There are several client-side scripting languages—JavaScript is the most widely used—that provide richer user interaction at the browser end.
- Complex applications usually have a multilayer architecture, including a model implementing business logic, a controller, and a view mechanism to display results. They may also include a data access layer that implements an object-relational mapping. Many applications implement and use web services, allowing functions to be invoked over HTTP.
- Techniques such as caching of various forms, including query result caching and connection pooling, and parallel processing are used to improve application performance.
- Application developers must pay careful attention to security, to prevent attacks such as SQL injection attacks and cross-site scripting attacks.
- SQL authorization mechanisms are coarse grained and of limited value to applications that deal with large numbers of users. Today application programs implement fine-grained, tuple-level authorization, dealing with a large number of application users, completely outside the database system. Database extensions to provide tuple-level access control and to deal with large numbers of application users have been developed, but are not standard as yet.

- Protecting the privacy of data are an important task for database applications. Many countries have legal requirements on protection of certain kinds of data, such as credit-card information or medical data.
- Encryption plays a key role in protecting information and in authentication of users and web sites. Symmetric-key encryption and public-key encryption are two contrasting but widely used approaches to encryption. Encryption of certain sensitive data stored in databases is a legal requirement in many countries and states.
- Encryption also plays a key role in authentication of users to applications, of Web sites to users, and for digital signatures.

Review Terms

- Application programs
- Web interfaces to databases
- HTML
- Hyperlinks
- Uniform resource locator (URL)
- Forms
- HyperText Transfer Protocol (HTTP)
- Connectionless protocols
- Cookie
- Session
- Servlets and Servlet sessions
- Server-side scripting
- Java Server Pages (JSP)
- PHP
- Client-side scripting
- JavaScript
- Document Object Model (DOM)
- Ajax
- Progressive Web Apps
- Application architecture
- Presentation layer
- Model-view-controller (MVC) architecture
- Business-logic layer
- Data-access layer
- Object-relational mapping
- Hibernate
- Django
- Web services
- RESTful web services
- Web application frameworks
- Connection pooling
- Query result caching
- Application security
- SQL injection
- Cross-site scripting (XSS)
- Cross-site request forgery (XSRF)
- Authentication
- Two-factor authentication
- Man-in-the-middle attack
- Central authentication
- Single sign-on
- OpenID
- Authorization
- Virtual Private Database (VPD)
- Audit trail

- Encryption
- Symmetric-key encryption
- Public-key encryption
- Dictionary attack
- Challenge – response
- Digital signatures
- Digital certificates

Practice Exercises

9.1 What is the main reason why servlets give better performance than programs that use the common gateway interface (CGI), even though Java programs generally run slower than C or C++ programs?

9.2 List some benefits and drawbacks of connectionless protocols over protocols that maintain connections.

9.3 Consider a carelessly written web application for an online-shopping site, which stores the price of each item as a hidden form variable in the web page sent to the customer; when the customer submits the form, the information from the hidden form variable is used to compute the bill for the customer. What is the loophole in this scheme? (There was a real instance where the loophole was exploited by some customers of an online-shopping site before the problem was detected and fixed.)

9.4 Consider another carelessly written web application which uses a servlet that checks if there was an active session but does not check if the user is authorized to access that page, instead depending on the fact that a link to the page is shown only to authorized users. What is the risk with this scheme? (There was a real instance where applicants to a college admissions site could, after logging into the web site, exploit this loophole and view information they were not authorized to see; the unauthorized access was, however, detected, and those who accessed the information were punished by being denied admission.)

9.5 Why is it important to open JDBC connections using the try-with-resources (`try (…){ … }`) syntax?

9.6 List three ways in which caching can be used to speed up web server performance.

9.7 The `netstat` command (available on Linux and on Windows) shows the active network connections on a computer. Explain how this command can be used to find out if a particular web page is not closing connections that it opened, or if connection pooling is used, not returning connections to the connection pool. You should account for the fact that with connection pooling, the connection may not get closed immediately.

9.8 Testing for SQL-injection vulnerability:

a. Suggest an approach for testing an application to find if it is vulnerable to SQL injection attacks on text input.

b. Can SQL injection occur with forms of HTML input other than text boxes? If so, how would you test for vulnerability?

9.9 A database relation may have the values of certain attributes encrypted for security. Why do database systems not support indexing on encrypted attributes? Using your answer to this question, explain why database systems do not allow encryption of primary-key attributes.

9.10 Exercise 9.9 addresses the problem of encryption of certain attributes. However, some database systems support encryption of entire databases. Explain how the problems raised in Exercise 9.9 are avoided if the entire database is encrypted.

9.11 Suppose someone impersonates a company and gets a certificate from a certificate-issuing authority. What is the effect on things (such as purchase orders or programs) certified by the impersonated company, and on things certified by other companies?

9.12 Perhaps the most important data items in any database system are the passwords that control access to the database. Suggest a scheme for the secure storage of passwords. Be sure that your scheme allows the system to test passwords supplied by users who are attempting to log into the system.

Exercises

9.13 Write a servlet and associated HTML code for the following very simple application: A user is allowed to submit a form containing a value, say n, and should get a response containing n "*" symbols.

9.14 Write a servlet and associated HTML code for the following simple application: A user is allowed to submit a form containing a number, say n, and should get a response saying how many times the value n has been submitted previously. The number of times each value has been submitted previously should be stored in a database.

9.15 Write a servlet that authenticates a user (based on user names and passwords stored in a database relation) and sets a session variable called *userid* after authentication.

9.16 What is an SQL injection attack? Explain how it works and what precautions must be taken to prevent SQL injection attacks.

9.17 Write pseudocode to manage a connection pool. Your pseudocode must include a function to create a pool (providing a database connection string, database user name, and password as parameters), a function to request a connection

from the pool, a connection to release a connection to the pool, and a function to close the connection pool.

9.18 Explain the terms CRUD and REST.

9.19 Many web sites today provide rich user interfaces using Ajax. List two features each of which reveals if a site uses Ajax, without having to look at the source code. Using the above features, find three sites which use Ajax; you can view the HTML source of the page to check if the site is actually using Ajax.

9.20 XSS attacks:

a. What is an XSS attack?

b. How can the referer field be used to detect some XSS attacks?

9.21 What is multifactor authentication? How does it help safeguard against stolen passwords?

9.22 Consider the Oracle Virtual Private Database (VPD) feature described in Section 9.8.5 and an application based on our university schema.

a. What predicate (using a subquery) should be generated to allow each faculty member to see only *takes* tuples corresponding to course sections that they have taught?

b. Give an SQL query such that the query with the predicate added gives a result that is a subset of the original query result without the added predicate.

c. Give an SQL query such that the query with the predicate added gives a result containing a tuple that is not in the result of the original query without the added predicate.

9.23 What are two advantages of encrypting data stored in the database?

9.24 Suppose you wish to create an audit trail of changes to the *takes* relation.

a. Define triggers to create an audit trail, logging the information into a relation called, for example, *takes_trail*. The logged information should include the user-id (assume a function *user_id*() provides this information) and a timestamp, in addition to old and new values. You must also provide the schema of the *takes_trail* relation.

b. Can the preceding implementation guarantee that updates made by a malicious database administrator (or someone who manages to get the administrator's password) will be in the audit trail? Explain your answer.

9.25 Hackers may be able to fool you into believing that their web site is actually a web site (such as a bank or credit card web site) that you trust. This may be done by misleading email, or even by breaking into the network infrastructure

and rerouting network traffic destined for, say mybank.com, to the hacker's site. If you enter your user name and password on the hacker's site, the site can record it and use it later to break into your account at the real site. When you use a URL such as https://mybank.com, the HTTPS protocol is used to prevent such attacks. Explain how the protocol might use digital certificates to verify authenticity of the site.

9.26 Explain what is a challenge – response system for authentication. Why is it more secure than a traditional password-based system?

Project Suggestions

Each of the following is a large project, which can be a semester-long project done by a group of students. The difficulty of the project can be adjusted easily by adding or deleting features.

You can choose to use either a web front-end using HTML5, or a mobile front-end on Android or iOS for your project.

Project 9.1 Pick your favorite interactive web site, such as Bebo, Blogger, Facebook, Flickr, Last.FM, Twitter, Wikipedia; these are just a few examples, there are many more. Most of these sites manage a large amount of data and use databases to store and process the data. Implement a subset of the functionality of the web site you picked. Implementing even a significant subset of the features of such a site is well beyond a course project, but it is possible to find a set of features that is interesting to implement yet small enough for a course project.

Most of today's popular web sites make extensive use of Javascript to create rich interfaces. You may wish to go easy on this for your project, at least initially, since it takes time to build such interfaces, and then add more features to your interfaces, as time permits.

Make use of web application development frameworks, or Javascript libraries available on the web, such as the jQuery library, to speed up your front-end development. Alternatively, implement the application as a mobile app on Android or iOS.

Project 9.2 Create a "mashup" which uses web services such as Google or Yahoo map APIs to create an interactive web site. For example, the map APIs provide a way to display a map on the web page, with other information overlaid on the maps. You could implement a restaurant recommendation system, with users contributing information about restaurants such as location, cuisine, price range, and ratings. Results of user searches could be displayed on the map. You could allow Wikipedia-like features, such as allowing users to add information and edit

information added by other users, along with moderators who can weed out malicious updates. You could also implement social features, such as giving more importance to ratings provided by your friends.

Project 9.3 Your university probably uses a course-management system such as Moodle, Blackboard, or WebCT. Implement a subset of the functionality of such a course-management system. For example, you can provide assignment submission and grading functionality, including mechanisms for students and teachers/teaching assistants to discuss grading of a particular assignment. You could also provide polls and other mechanisms for getting feedback.

Project 9.4 Consider the E-R schema of Practice Exercise 6.3 (Chapter 6), which represents information about teams in a league. Design and implement a web-based system to enter, update, and view the data.

Project 9.5 Design and implement a shopping cart system that lets shoppers collect items into a shopping cart (you can decide what information is to be supplied for each item) and purchased together. You can extend and use the E-R schema of Exercise 6.21 of Chapter 6. You should check for availability of the item and deal with nonavailable items as you feel appropriate.

Project 9.6 Design and implement a web-based system to record student registration and grade information for courses at a university.

Project 9.7 Design and implement a system that permits recording of course performance information—specifically, the marks given to each student in each assignment or exam of a course, and computation of a (weighted) sum of marks to get the total course marks. The number of assignments/exams should not be predefined; that is, more assignments/exams can be added at any time. The system should also support grading, permitting cutoffs to be specified for various grades.

You may also wish to integrate it with the student registration system of Project 9.6 (perhaps being implemented by another project team).

Project 9.8 Design and implement a web-based system for booking classrooms at your university. Periodic booking (fixed days/times each week for a whole semester) must be supported. Cancellation of specific lectures in a periodic booking should also be supported.

You may also wish to integrate it with the student registration system of Project 9.6 (perhaps being implemented by another project team) so that classrooms can be booked for courses, and cancellations of a lecture or addition of extra lectures can be noted at a single interface and will be reflected in the classroom booking and communicated to students via email.

Project 9.9 Design and implement a system for managing online multiple-choice tests. You should support distributed contribution of questions (by teaching assistants, for example), editing of questions by whoever is in charge of the course, and creation of tests from the available set of questions. You should also be able to administer tests online, either at a fixed time for all students or at any time but with a time limit from start to finish (support one or both), and the system should give students feedback on their scores at the end of the allotted time.

Project 9.10 Design and implement a system for managing email customer service. Incoming mail goes to a common pool. There is a set of customer service agents who reply to email. If the email is part of an ongoing series of replies (tracked using the in-reply-to field of email) the mail should preferably be replied to by the same agent who replied earlier. The system should track all incoming mail and replies, so an agent can see the history of questions from a customer before replying to an email.

Project 9.11 Design and implement a simple electronic marketplace where items can be listed for sale or for purchase under various categories (which should form a hierarchy). You may also wish to support alerting services, whereby a user can register interest in items in a particular category, perhaps with other constraints as well, without publicly advertising her interest, and is notified when such an item is listed for sale.

Project 9.12 Design and implement a web-based system for managing a sports "ladder." Many people register and may be given some initial rankings (perhaps based on past performance). Anyone can challenge anyone else to a match, and the rankings are adjusted according to the result. One simple system for adjusting rankings just moves the winner ahead of the loser in the rank order, in case the winner was behind earlier. You can try to invent more complicated rank-adjustment systems.

Project 9.13 Design and implement a publication-listing service. The service should permit entering of information about publications, such as title, authors, year, where the publication appeared, and pages. Authors should be a separate entity with attributes such as name, institution, department, email, address, and home page.

Your application should support multiple views on the same data. For instance, you should provide all publications by a given author (sorted by year, for example), or all publications by authors from a given institution or department. You should also support search by keywords, on the overall database as well as within each of the views.

Project 9.14 A common task in any organization is to collect structured information from a group of people. For example, a manager may need to ask employees to enter their vacation plans, a professor may wish to collect feedback on a particu-

lar topic from students, or a student organizing an event may wish to allow other students to register for the event, or someone may wish to conduct an online vote on some topic. Google Forms can be used for such activities; your task is to create something like Google Forms, but with authorization on who can fill a form.

Specifically, create a system that will allow users to easily create information collection events. When creating an event, the event creator must define who is eligible to participate; to do so, your system must maintain user information and allow predicates defining a subset of users. The event creator should be able to specify a set of inputs (with types, default values, and validation checks) that the users will have to provide. The event should have an associated deadline, and the system should have the ability to send reminders to users who have not yet submitted their information. The event creator may be given the option of automatic enforcement of the deadline based on a specified date/time, or choosing to login and declare the deadline is over. Statistics about the submissions should be generated—to do so, the event creator may be allowed to create simple summaries on the entered information. The event creator may choose to make some of the summaries public, viewable by all users, either continually (e.g., how many people have responded) or after the deadline (e.g., what was the average feedback score).

Project 9.15 Create a library of functions to simplify creation of web interfaces, using jQuery. You must implement at least the following functions: a function to display a JDBC result set (with tabular formatting), functions to create different types of text and numeric inputs (with validation criteria such as input type and optional range, enforced at the client by appropriate JavaScript code), and functions to create menu items based on a result set. Also implement functions to get input for specified fields of specified relations, ensuring that database constraints such as type and foreign-key constraints are enforced at the client side. Foreign key constraints can also be used to provide either autocomplete or drop-down menus, to ease the task of data entry for the fields.

For extra credit, use support CSS styles which allow the user to change style parameters such as colors and fonts. Build a sample database application to illustrate the use of these functions.

Project 9.16 Design and implement a web-based multiuser calendar system. The system must track appointments for each person, including multioccurrence events, such as weekly meetings, and shared events (where an update made by the event creator gets reflected to all those who share the event). Provide interfaces to schedule multiuser events, where an event creator can add a number of users who are invited to the event. Provide email notification of events. For extra credits implement a web service that can be used by a reminder program running on the client machine.

Tools

There are several integrated development environments that provide support for web application development. Eclipse (www.eclipse.org) and Netbeans (netbeans.org) are popular open-source IDEs. IntelliJ IDEA (www.jetbrains.com/idea/) is a popular commercial IDE which provides free licenses for students, teachers and non-commercial open source projects. Microsoft's Visual Studio (visualstudio.microsoft.com) also supports web application development. All these IDEs support integration with application servers, to allow web applications to be executed directly from the IDE.

The Apache Tomcat (jakarta.apache.org), Glassfish (javaee.github.io/glassfish/), JBoss Enterprise Application Platform (developers.redhat.com/products/eap/overview/), WildFly (wildfly.org) (which is the community edition of JBoss) and Caucho's Resin (www.caucho.com), are application servers that support servlets and JSP. The Apache web server (apache.org) is the most widely used web server today. Microsoft's IIS (Internet Information Services) is a web and application server that is widely used on Microsoft Windows platforms, supporting Microsoft's ASP.NET (msdn.microsoft.com/asp.net/).

The jQuery JavaScript library jquery.com is among the most widely used JavaScript libraries for creating interactive web interfaces.

Android Studio (developer.android.com/studio/) is a widely used IDE for developing Android apps. XCode (developer.apple.com/xcode/) from Apple and AppCode (www.jetbrains.com/objc/) are popular IDEs for iOS application development. Google's Flutter framework (flutter.io), which is based on the Dart language, and Facebook's React Native (facebook.github.io/react-native/) which is based on Javascript, are frameworks that support cross-platform application development across Android and iOS.

The Open Web Application Security Project (OWASP) (www.owasp.org) provides a variety of resources related to application security, including technical articles, guides, and tools.

Further Reading

The HTML tutorials at www.w3schools.com/html, the CSS tutorials at www.w3schools.com/css are good resources for learning HTML and CSS. A tutorial on Java Servlets can be found at docs.oracle.com/javaee/7/tutorial/servlets.htm. The JavaScript tutorials at www.w3schools.com/js are an excellent source of learning material on JavaScript. You can also learn more about JSON and Ajax as part of the JavaScript tutorial. The jQuery tutorial at www.w3schools.com/Jquery is a very good resource for learning how to use jQuery. These tutorials allow you to modify sample code and test it in the browser, with no software download. Information about the

.NET framework and about web application development using ASP.NET can be found at msdn.microsoft.com.

You can learn more about the Hibernate ORM and Django (including the Django ORM) from the tutorials and documentation at hibernate.org/orm and docs.djangoproject.com respectively.

The Open Web Application Security Project (OWASP) (www.owasp.org) provides a variety of technical material such as the OWASP Testing Guide, the OWASP Top Ten document which describes critical security risks, and standards for application security verification.

The concepts behind cryptographic hash functions and public-key encryption were introduced in [Diffie and Hellman (1976)] and [Rivest et al. (1978)]. A good reference for cryptography is [Katz and Lindell (2014)], while [Stallings (2017)] provides textbook coverage of cryptography and network security.

Bibliography

[Diffie and Hellman (1976)] W. Diffie and M. E. Hellman, "New Directions in Cryptography", *IEEE Transactions on Information Theory*, Volume 22, Number 6 (1976), pages 644–654.

[Katz and Lindell (2014)] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 3rd edition, Chapman and Hall/CRC (2014).

[Rivest et al. (1978)] R. L. Rivest, A. Shamir, and L. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems", *Communications of the ACM*, Volume 21, Number 2 (1978), pages 120–126.

[Stallings (2017)] W. Stallings, *Cryptography and Network Security - Principles and Practice*, 7th edition, Pearson (2017).

Credits

The photo of the sailboats in the beginning of the chapter is due to ©Pavel Nesvadba/Shutterstock.